

Computer Security

Vrije Universiteit Amsterdam - Faculty of Science - Computer Security - 2025-2026

If you want to reach the top of the field of Computer Security, CSec is your program. Our Master's program in Computer Security is unique in the Netherlands and one of the few of its kind in the world. Created by world-class researchers, the program addresses the most ambitious technical challenges in Cybersecurity, with a focus on computer systems and networks security and topics such as hacking, side channels, hardening, verification, malware, vulnerability analysis, and reverse engineering. The emphasis is on experimental computer science rather than theory. As such, most of the courses in the curriculum are very hands-on, which means you will get extensive practical and research-oriented experience in computer systems and security.

You will study the inner workings of modern computer systems and understand how they have become increasingly complex to meet the challenges of Clouds, Big Data, the Internet of Things, etc. and the ever-growing demand for better performance. You will learn how such sheer complexity comes at a nontrivial security cost, with each layer of the modern hardware and software stack plagued by critical security vulnerabilities.

During your studies, you will learn how to use experimental and empirical approaches to understand and address today's systems security issues towards tomorrow's hacker-proof computer systems. What's more, you will design, implement, and evaluate complex systems and security solutions. To this end, you will often work in small teams to learn valuable collaborative research skills.

Upon completion of your studies, you'll be prepared for top positions in industry, with high-demand roles such as security researcher, analyst, or lead system designer/architect. After graduating, you could also continue your studies with a PhD research position at a top university. Or, you could join one of the excellent research labs in industry.

Info

Level	Master
Language	English
Duration	2 years
Type	Full Time
Credits	120 EC
Faculty	Faculty of Science

Teaching and examination regulation 2025-2026
More information about this programme

Computer Security year 1 compulsory courses 4
Computer Security year 2 compulsory courses 4
Computer Security constrained choice FCC 4
Computer Security electives 4
Courses 5
Advanced Algorithms 5
Advanced Compiler Construction 5
Advanced Logic 7
Advanced Operating Systems 8
AI for Security 10
Binary and Malware Analysis 12
Cryptographic Engineering 13
Data Mining Techniques 14
Digital Architecture 15
Distributed Algorithms 17
Distributed Systems 18
Dynamic Programming and Reinforcement Learning 20
Evolutionary Computing 21
Experimental Design and Data Analysis 22
Foundations of Cryptography 23
Green Lab 24
Hardware Security 26
High Performance Computing and Big Data 27
Introduction to Computational Science 27
Large Research Project in Computer Security 28
Machine Learning for the Quantified Self 29
Master Project Computer Security 31
Network Security 32
Parallel Programming Practical 33
Performance of Networked Systems 34
Programming Large-Scale Parallel Systems 36
Research Project in Computer Security 37
Secure and Trustworthy Machine Learning 38
Security Experiments and Measurements 40
Service Oriented Design 43
Software Security 45
Software Testing 47
Systems Seminar 48
The Social Web 50
Verification for Security 51
Web Services and Cloud-Based Systems 52

Computer Security year 1 compulsory courses

Courses

Course Name	Period	Credits	Code
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Software Security	P1	6EC	X_400127
Systems Seminar	P4	6EC	XM_0122
Binary and Malware Analysis	P5	6EC	X_405100

Computer Security year 2 compulsory courses

Courses

Course Name	Period	Credits	Code
Master Project Computer Security	Ac. Year (September)	30EC	XM_0123
Advanced Operating Systems	P1	6EC	XM_40014
Hardware Security	P2	6EC	XM_40019

Computer Security constrained choice FCC

Description

Students must complete at least one course of 6 EC from those listed below.

Note: Every study plan, including the selection of constrained choice and elective courses, has to be discussed with and agreed upon by the master coordinator or a personal mentor and approved by the Examination Board.

Courses

Course Name	Period	Credits	Code
Verification for Security	P1	6EC	XM_0099
Advanced Algorithms	P2	6EC	XMU_0060
Foundations of Cryptography	P2	6EC	XM_0174
Advanced Logic	P4	6EC	X_405048
Distributed Algorithms	P5	6EC	X_400211

Computer Security electives

Courses

Course Name	Period	Credits	Code
Large Research Project in Computer Security	Ac. Year (September)	12EC	XM_0126
Research Project in Computer Security	Ac. Year (September)	6EC	XM_0125
Evolutionary Computing	P1	6EC	X_400111
Green Lab	P1	6EC	X_418158
Introduction to Computational Science	P1	6EC	XMU_418111
Service Oriented Design	P1	6EC	X_405061
Digital Architecture	P2	6EC	XM_0127
Distributed Systems	P2	6EC	X_400130
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Network Security	P2	6EC	XM_0100
Performance of Networked Systems	P2	6EC	X_405105

Course Name	Period	Credits	Code
The Social Web	P2	6EC	X_405086
Advanced Compiler Construction	P3	6EC	XM_0139
High Performance Computing and Big Data	P3	6EC	XMU_40013
Parallel Programming Practical	P3	6EC	X_400162
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Cryptographic Engineering	P4	6EC	XMU_0047
Experimental Design and Data Analysis	P4	6EC	X_405078
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Data Mining Techniques	P5	6EC	X_400108
Software Testing	P5	6EC	X_400439
AI for Security	P6	6EC	XM_0178
Machine Learning for the Quantified Self	P6	6EC	XM_40012

Courses

Advanced Algorithms

Course Code	XMU_0060
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130403

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Advanced Compiler Construction

Course Code	XM_0139
Credits	6
Period	P3

Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. K. Freiherr von Gleissenthal
Examiner	dr. K. Freiherr von Gleissenthal
Teaching Staff	prof. dr. C. Giuffrida, dr. K. Freiherr von Gleissenthal
Teaching method(s)	Seminar, Practical, Lecture

Course Objective

Upon completion of this course, students will:

- have knowledge of grammars and compiler frontend principles, such as lexing, (topdown and bottom-up) parsing, abstract syntax trees, semantic analysis, and intermediate representation (IR) generation (K&U, AK&U, MJ)
- have a basic understanding of the compiler design and implementation principles adopted by mainstream compiler frameworks such as LLVM (K&U)
- have knowledge of compiler middle-end performance principles such as IR-level optimizations and experience implementing them (K&U, AK&U, MJ)
- have knowledge of compiler middle-end security principles such as IR-level security hardening and experience implementing them (K&U, AK&U, MJ)
- have knowledge of compiler backend performance principles such as machine-level optimizations (K&U, AK&U)
- have knowledge of compiler backend security principles such as machine-level security hardening (K&U, AK&U, MJ)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This course gives an introduction to the internals of modern compilers with a unique twist: the emphasis is not just on translating a source program into an executable binary, or traditional optimization passes, but also on software hardening techniques to enhance the target program’s security. As a result, the student learns about all fundamental aspects of compilers, but also the performance and security implications of the code it generates. To address these concerns, students learn compiler-based techniques that improve both the performance and the security. The course allows students to gather practical, hands-on experience on building compiler components with weekly assignments.

The course focuses on the design of the major components of a modern compiler pipeline: frontend (focus on lexing and parsing), intermediate representation or IR (focus on optimizations, transformations, and hardening), and backend (focus on code generation). In both the lectures and the assignments, we emphasize what compiler builders encounter in practice, rather than the details of all manners of parsing algorithms. Specifically, the assignments focus on building a compiler for a C-like language using the LLVM compiler framework, analyzing the code so that we can optimize it, and adding additional checks to eliminate common vulnerabilities that are common to C-like languages. All the assignments build on top of a given framework and require localized extensions in Python and/or C++ to implement language features, optimizations, transformations/instrumentations, and code generation features.

Note: this is a very intensive, hands-on course. It is important to be able to start immediately. For this, you need access to a computer with the VirtualBox x86 virtualization environment installed.

Additional Information Teaching Methods

Lectures, Practical.

Method of Assessment

Written exam on the material covered in lectures and the (team) programming assignments. The final grade is the weighted average of the exam grade (50%) and the assignments (50%). To pass the course, the exam grade must be at least 5.5, each assignment grade must be at least 4.0, and the final grade must be at least 5.5.

Please note that your grade will be registered as follows:

- If you do not submit any assignment and do not participate in the exam, you will receive a NS.
- In all other cases, you will receive either a numeric grade or incomplete (NVD).

- If you did not meet the minimum grade requirements for the assignments, your grade is NVD.
- Otherwise, your final grade is your weighted average.
- If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

There is no resit opportunity for the practical assignments. Results for practical assignments and for the exam may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner). If you submit an assignment or participate in the exam, the new result replaces any transferred results.

We ensure that each student individually demonstrates their abilities through the requirement that the supervised exam be passed with at least 5.5, through active plagiarism detection for the assignments, and by investigating whether each team member contributed sufficiently to the team assignments.

Note: this course has two variants (“Compiler Construction” and “Advanced Compiler Construction”) offered to two different target groups (“Bachelor CS”) and (“Master CSec”). At the Master level, there are four mandatory assignments. At the Bachelor level, there are only three mandatory assignments. For the assignments, students can only work in teams with other students of the same target group.

Literature

No set book. All material will be made available during the course.

Additional Information Target Audience

Bachelor Computer Science (year 3) / Master Computer Security (year 1)

Recommended background knowledge

Knowledge of Python and C/C++ is essential.

Advanced Logic

Course Code	X_405048
Credits	6
Period	P4
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. van Raamsdonk
Examiner	dr. F. van Raamsdonk
Teaching Staff	dr. F. van Raamsdonk
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

The objective of the course Advanced Logic is to obtain a basic understanding of modal logic.

Knowledge and insight:

- know the Kripke semantics, and variations on that definition, for example game semantics,
- know the tableaux and sequent method for decidability,
- know the notion of bisimulation, and the game approach to (dis)proving bisimularity,
- know the nature of Hilbert style proof systems,
- know basic temporal logic and multi-model logic,
- know propositional dynamic logic and its semantics;

Applying knowledge and insight:

- be able to give proofs using Kripke semantics, game semantics,
- be able to decide on (non)validity using the tableaux or sequents method,
- be able to (dis)prove bisimilarity of two pointed models also in a game approach,
- be able to make elementary derivations in Hilbert style systems,
- be able to give proofs and show (non-)bisimilarity for basic temporal logic and multi-modal logic,
- be able to (dis)prove validity and discuss models in propositional dynamic logic;

Judgement:

- be able to (dis)prove definability of operators and frame properties in basic modal logic and also in basic temporal logic and multi-modal logic,
- be able to discuss some decidability issues;

Communication:

- be able to write elementary proofs;

Learning skills:

- have acquired some flexibility in dealing with different versions of modal logic.

Course Content

The course Advanced Logic is concerned with modal logic and its applications to computer science. We study some themes from the book Modal Logics for Open Minds by Johan van Benthem, in particular: basic modal logic and possible world semantics, bisimulation and invariance, modal definability and expressive power, decidability, proof systems, propositional dynamic logic, some epistemic logic.

Additional Information Teaching Methods

We have two lectures per week, for the duration of seven weeks. Twelve lectures discuss material and two lectures are meant for questions and wrap-up.
We have one exercise class per week, for the duration of seven weeks. We discuss (on paper) exercises.

Method of Assessment

There is a written closed-book midterm exam about the first half of the material in week 4 of the course.
There is a written closed-book final exam about all material in week 8 of the course.

If the grade for the midterm exam is higher than the grade for the final exam, then the midterm counts for 20% of the final grade (midterm-grade + 4*final-exam-grade)/5), otherwise the final grade is the grade of the final exam.

Literature

- Johan van Benthem, Modal Logics for Open Minds, CSLI Publications 2010.
- In addition we use slides and exercise sheets.

Additional Information Target Audience

MSc Artificial Intelligence
MSc Computer Science
MSc Computer Security

Explanation Canvas

We use Canvas for the schedule of the course, and to make the slides for the lectures and the exercises for the exercise classes available.
We use Canvas further to provide feedback to the homework assignments, to give the grade for the midterm, and for announcements.

Recommended background knowledge

Basic knowledge of the syntax and semantics of first-order proposition and predicate logic, as for example taught in the Bachelor course Logic and Modelling (X_401015).

Advanced Operating Systems

Course Code	XM_40014
Credits	6
Period	P1
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. C. Giuffrida
Examiner	prof. dr. C. Giuffrida

Teaching Staff	dr. E. van der Kouwe, prof. dr. C. Giuffrida
Teaching method(s)	Lecture

Course Objective

Gaining a deep understanding of design and implementation of major OS subsystems.

Upon completion of this course, students will:

- have knowledge of advanced operating system design and implementation principles adopted by mainstream operating systems such as Linux (Knowledge & Understanding)
- have a basic understanding of recent trends in operating systems research (Lifelong learning skills)
- be familiar with design and implementation principles of physical memory management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of page table management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of process management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of virtual memory management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of multiprocessing (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of multicore management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of page reclaiming (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)

Course Content

The course will feature a number of hands-on assignments accompanied by lectures on advanced operating system kernel design and programming concepts. In each assignment, students will be expected to start with a minimal kernel implementation and exercise their kernel hacking skills on one of the major operating subsystems (i.e., memory management, process management, drivers, etc.). This will involve programming in both C and assembly as well as directly interfacing with the hardware. The course will also link lectures and assignments to modern operating system features and offer insights into state-of-the-art OS research efforts.

Additional Information Teaching Methods

Lectures and practical assignments.

Method of Assessment

Weekly examination based on practical assignments and lectures. Each week, the student can get up to 11 points for the weekly examination: 6 points from a practical assignment, 2 points for the bonus parts of the assignment, and 3 points from an oral discussion that is based on the assignment and its companion lectures for that week. In each of the first six weeks, you need at least 50% of the points for the oral discussion (at least 1.5 points), along with passing all basic compliance tests for the practical assignment. The seventh (and last) practical assignment is special: the oral discussion is replaced by a written report and there is no minimum grade to pass the assignment. The final grade is the average of all weekly examinations' grades (including the last practical assignment grade, which is considered to be zero if it was not submitted).

Please note that your final grade will be registered as follows:

- If you do not submit any assignment and do not participate in any oral discussion, you will receive a NS.
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments, your grade is NVD.
- Otherwise, your final grade is your weighted average.
- If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

There is no resit opportunity for the practical assignments, nor for the oral discussions. Results for practical assignments and for the oral discussion may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner). If you submit an assignment or participate in the oral discussion, the new result replaces any transferred results.

We ensure that each student individually demonstrates their abilities through oral exams to test understanding of the assignments, through active plagiarism detection for the assignments, and by investigating whether each group member contributed sufficiently for the group assignment.

Literature

Slides and online material.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

Students are expected to have completed a Bachelor-level Operating systems course as well as to have experience in C/C++ programming and knowledge of assembly and computer organization.

AI for Security

Course Code	XM_0178
Credits	6
Period	P6
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. M. Zhang
Examiner	dr. M. Zhang
Teaching Staff	M. Chen, dr. M. Zhang
Teaching method(s)	Seminar, Lecture

Course Objective

This course explores a range of topics at the intersection of security and artificial intelligence (AI). The primary goal is to develop students' ability to think critically about how AI techniques can be effectively applied to address real-world security problems.

Upon completion of this course, the students will:

- Understand several security problems, e.g., software vulnerability mining, vulnerability assessment, anomaly detection (K&U, LS, C)
- Understand AI techniques, e.g., neural networks, large language models (K&U, LS)
- Have knowledge of example(s) about how to apply AI to a domain-specific problem, e.g., model security events into sequential data, connect security incidents as graphs (AK&U, MJ, LS)
- Have developed critical thinking around AI and security (MJ, LS, C)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This course explores a range of rapidly evolving security topics that have seen significant advances through the application of artificial intelligence (AI) techniques. Topics may include, but are not limited to, AI-assisted software development, automated bug discovery, automated software repair, and vulnerability assessment.

The course is structured around concrete security problems. For each topic, we begin by introducing the core security challenge, then examine how it has been traditionally addressed. We subsequently explore how AI techniques have been applied to enhance or automate solutions to the problem. Finally, we discuss the open challenges and limitations that must be addressed before AI can be effectively applied to new or emerging security scenarios.

To understand state-of-the-art methods, the course focuses on the discussion of scientific papers published at top-tier venues. Some topics may be accompanied by practical implementations; therefore, programming skills are expected.

Additional Information Teaching Methods

The course is organized into weekly lectures accompanied by practical quizzes and one final report based on a

selection of five research papers. A list of papers will be provided, from which each student will choose five. The report should present a critical analysis of a specific security issue addressed in the selected papers, comparing the strengths and limitations of the approaches used. Students are expected to demonstrate critical thinking by identifying open challenges and proposing possible future directions related to the problem.

Peer Review of Reports:

Students will participate in a peer-review process by evaluating reports submitted by their classmates. Each student will provide a detailed assessment based on predefined criteria, including the quality of analysis, clarity of writing, comparison of methods, and depth of critical thinking.

Self-Evaluation of Report:

In addition to peer review, students will complete a self-evaluation of their own report. This will involve providing a detailed self-assessment against the same grading criteria, encouraging reflection on their work and the learning process.

Method of Assessment

The overall grade is determined as follows:

- Quizzes (30%)

Three quizzes, each worth 10%.

- Report (20%)

Report content: 15%, assessed based on depth of analysis, clarity, comparison of methods, and critical insight.
Peer review quality: 5%, based on the thoroughness and constructiveness of the peer reviews provided.

- Final Exam (50%)

A comprehensive exam covering the full course content.

Pass or fail
Quality of the reviews. Additional points will be deducted from students who write poor (e.g., too generic, such as "it is good") or unfair reviews.

To pass the course, you must satisfy the following criteria:

- You must score at least 6.0 on the final exam, and
- You must score 6.0 in total (i.e., with all three components taken together).

Resit
There is a resit for the final exam (50%). You will have to score 6.0 to pass the resit.
It is ***not possible*** to resit the assignments and the peer reviews.

Please note that your grade will be registered as follows:

For the first exam date: if you did not submit any quizzes, report, and did not participate in the exam, the result will be registered as no show (NS). For the second exam date: if you did not participate in the resit exam, the result will be registered as no show (NS). In all other cases, you will receive either a numeric grade or incomplete (NVD).

If you did not meet the minimum grade requirements for the exam (6.0), but your weighted average is >= 6.0, your grade is NVD. Otherwise, your final grade is your weighted average, rounded to half points except for 6.0.

If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

Entry Requirements

While there are no formal entry requirements, however, students are expected to have foundational knowledge in either cybersecurity or artificial intelligence to fully benefit from the material.

Literature

No set book. All materials (i.e., papers, slides) will be made available during the course.

Additional Information Target Audience

Master Computer Security
Master Computer Science

Master Artificial Intelligence

Additional Information

Please see the information on Canvas.

Explanation Canvas

The submission of the report and the peer review of the report will be done in Canvas/FeedbackFruit.

Because of the way the system works by automatically assigning reviewers, it is not possible to grant any deadline extension for the submission of the reports or the submission of the reviews.

Students should therefore carefully plan their activities and submit also a draft version of their reports well in advance of the deadline (multiple submissions are possible) rather than waiting for the last moment.

Recommended background knowledge

While there are no formal entry requirements, however, students are expected to have foundational knowledge in either cybersecurity or artificial intelligence to fully benefit from the material. Programming skills are also expected.

Binary and Malware Analysis

Course Code	X_405100
Credits	6
Period	P5
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. ir. H.J. Bos
Examiner	prof. dr. ir. H.J. Bos
Teaching Staff	dr. J.M. Slowinska, prof. dr. ir. H.J. Bos
Teaching method(s)	Practical, Lecture

Course Objective

Gain insights in static and dynamic analysis, applied to binaries and malware, and tackle fun challenges along the way.

Upon completion of this course, students will:

- have knowledge of state-of-the-art techniques for static and dynamic binary analysis (K&U)
- have sufficient knowledge and experience to reverse engineer the behavior of real-world benign or malicious program binaries (LS, AK&U)
- be able to recognize and overcome anti-analysis techniques in real binaries (K&U, AK&U, MJ)
- be able to recognize and overcome unpacking behavior in real binaries (K&U, AK&U, MJ)
- be able to perform static control-flow and data-flow analysis for real binaries (K&U, AK&U, MJ)
- be able to perform dynamic data-flow tracking for real binaries (K&U, AK&U, MJ)
- be able to perform binary lifting and symbolic analysis for real binaries (K&U, AK&U, MJ)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

Binaries in general, and malware in particular, are very hard to analyze. Unlike with source code, you have no idea what the binary does, or even what the data structures look like - let alone what they mean! Security analysts, forensic experts, and reverse engineers often have to dig their way through such programs to figure out what the code is all about, and where the interesting pieces of information are.

How do they do this? What techniques and tools can they fall back on, and, conversely, what techniques do the malware authors use to prevent this?

This is a hands-on specialization course for highly motivated students, who will learn essential analysis techniques and methods in both static and dynamic analysis. Not only will they pick apart a malicious binary, they will also be

working on a set of cool and very complicated challenges to find a secret buried deep inside a binary program.

During the course we will cover everything from static analysis (CFG analysis, reverse-engineering, deobfuscation, etc), dynamic analysis (e.g., taint analysis), symbolic execution, static and dynamic patching, and many other techniques for pulling apart a mysterious binary. In the assignments, students will be exposed to programs that actively fight static and dynamic analysis. To circumvent the binary's defenses, students will use binary patching. After studying the details of popular binary formats (ELF, PE, etc.), they will patch the binary -- while adhering to the format. In addition, they will work with all manner of state-of-art system tools to lift and analyze the binaries (such as Ghidra, Intel PIN, etc.).

Additional Information Teaching Methods

Lectures and practicals.

Method of Assessment

The final grade is determined by the quality and correctness of solutions to the individual assignments and oral discussions about them. Each of the 5 practical assignments must be passed with a grade >= 4. The final grade is a weighted average of the assignments' grades, and the course can be passed with the final grade of at least 5.5.

There is no resit opportunity for the practical assignments, nor for the oral discussions.

We ensure that each student individually demonstrates their abilities through active plagiarism detection for the assignments, and by having an oral discussion with each student to verify that they can explain how they addressed the practical assignments.

Literature

No set book. All material will be made available during the course.

Suggested supplementary reading material: Andriesse, D, “Practical Binary Analysis: Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly”, No starch press, 2018.

Additional Information Target Audience

Master Computer Security

Recommended background knowledge

Systems programming, x86 assembly.

Cryptographic Engineering

Course Code	XMU_0047
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2024-2025-en/search-course/course/119769

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Data Mining Techniques

Course Code	X_400108
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. M. Hoogendoorn
Examiner	prof. dr. M. Hoogendoorn
Teaching Staff	dr. S. Yu, prof. dr. M. Hoogendoorn, B. Borsboom
Teaching method(s)	Computer lab, Lecture

Course Objective

The aim of the course is that students acquire data mining knowledge and skills that they can apply in a business environment. More precisely, the following learning goals are distinguished:

1. Understand the data mining process
2. Understand exploratory data analysis
3. Understand feature engineering
4. Understand classification techniques
5. Understand regression techniques
6. Understand association rules and recommender systems
7. Understand big data engineering and big data infrastructures
8. Understand current state of the art in data mining
9. Understand ethical concerns and evaluation of results
10. Learning to argue for choices (rationale)

The course if focused on the Dublin descriptors Applying Knowledge and Understanding (since the aim is to improve the practical skills); Making judgements (which technique is appropriate, how to best apply Data Mining for a specific case); Communication skills (how to report on your approach, choices and your results), and Learning skills (able to find new relevant techniques, assess their suitability, etc.).

How the aims are to be achieved: Students will acquire knowledge and skills mainly through the following: an overview of the most common data mining algorithms and techniques (in lectures), a survey of typical and interesting data mining applications, and practical assignments to gain "hands on" experience. The application of skills in a business environment will be simulated through various assignments of the course.

Course Content

The course is intended to introduce Data Mining Techniques to students that are new to the field as well as to more experienced students. The main aim is to gain a more practical perspective towards Data Mining Techniques/Machine Learning. Lectures will cover more basic things for those new to the field (general introduction into Data Mining, classical algorithms such as decision trees, association rules, neural networks, ensemble learning, etc.) and on top will discuss advanced topics including deep learning, recommender systems, big data infrastructures, and text mining. A number of successful applications in the area will also be discussed. In addition to lectures, there will be an extensive practical part, where students will experiment with various data mining algorithms and data sets. The grade for the course will be based on these practical assignments.

Additional Information Teaching Methods

Lectures (h) and practical sessions (pra). Lectures are planned to be interactive: there will be small questions, etc.

Method of Assessment

Practical assignments. There will be two assignments done in groups of three. For the first assignment there is a choice: going for a basic version of the assignment (suited for those new to the domain) or a more advanced one (for students with more experience). The second assignment is the same for all and will involve an in-class Kaggle competition. The first assignment counts for 40% and the second for 60%.

An interview will take place on an individual student level to make sure that (1) each student understands the content of the graded material, and (2) has made a sufficient contribution to the group work. Only when this check is passed will a final grade be given. Furthermore, the grade of both assignments needs to be sufficient to pass the course. Finally, attendance of the final lecture (in which the final ranking of the Kaggle competition is announced and selected groups of students will be asked to present their solution) is mandatory as it contributes to the learning objectives of the course.

There is an option to resit one of the assignments when the assignment has been submitted but an insufficient grade is obtained (only when the other assignment grade is sufficient). This involves improving your assignment to at least a sufficient level within two weeks after having received your grade for the second assignment. The maximum grade of the resit is a 6.0.

Literature

Optional: Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal. Data Mining: Practical Machine Learning Tools and Techniques (Fourth Edition). Morgan Kaufmann, 2016 978-0128042915.

Additional Information Target Audience

Master Artificial Intelligence, Master Computer Science, Master Bioinformatics and Systems Biology, Master Business Analytics

Recommended background knowledge

1. Statistics
2. Programming Skills

Digital Architecture

Course Code	XM_0127
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. R.C. de Boer
Examiner	dr. R.C. de Boer
Teaching Staff	dr. R.C. de Boer, dr. R.C. de Boer
Teaching method(s)	Seminar, Lecture

Course Objective

Knowledge and understanding
At the end of the course, students will have gained knowledge about architecturally significant requirements, architecture representation, design, quality and analysis. They understand the drivers behind architectural decisions and the ‘wicked’ nature of architecture design, and appreciate the role of the architect.

Applying knowledge and understanding
Students will be able to reason about the architecture of non-trivial software-intensive systems, and apply their knowledge and understanding to a concrete (fictitious) case that involves many stakeholders with different needs and concerns.

Making judgments
Students will make their own architectural decisions, and become aware of the rationale involved in making trade-offs and choosing design options among alternatives. They learn to appreciate the design decisions proposed by others. In a serious game setting, students will further experience architectural design decision making.

Communication skills
Students present their progress to the other course participants, and engage in debate about their architecture design. They communicate their final architecture design, including the architectural decisions and their rationale, in a document.

Learning skills
Students have to coordinate and negotiate team work, and manage a significant workload in the allotted time. Through searching and integrating background material, students build a progressive understanding of the ‘problem domain’ relevant to the case.

Course Content

Students work in groups to design an architecture for a fictitious software-intensive system. They have to develop different representations (called views) of the architecture. These different representations emphasize different concerns of people or organizations that have a stake in the system. The group work is complemented by theory lectures and in-depth sessions.

Additional Information Teaching Methods

Teaching methods and contact hours:

- Lectures: two hours per week
- Group work with a number of assignments; two contact hours per week
- Tutorials and in-depth sessions: two hours per week.

Attendance of the first lecture is compulsory! Students are required to participate in group work. All teams must be present at their scheduled contact hours to present their progress and engage in debate with the other teams.

Note that this course is fully taught on campus. Remote participation is not possible.

Method of Assessment

Graded assignments

Project assignments (group assignment): There are two assignments based on a practical case. These assignments are done as group work. The average grade of the assignments should be ≥ 5.5 and will account for 75% of the final grade. Your work will be evaluated by means of a rubric published on Canvas.

Written exam (individual assignment): To complete the course you will have to take part in the written exam for Digital Architecture. The exam takes place in the exam week at the end of the period. Your grade for the exam should be ≥ 5.5 and will account for 25% of your final grade.

Pass/fail-assigments

Participation in the serious game and Game survey (individual assignment): In-person participation in the game is compulsory. The game will be played by all students simultaneously on campus. After playing the architecture design decision-making game, each student should fill out the game survey individually. There will be only one opportunity to play the game, it is not possible to retake this assignment on another occasion. The assignment will be graded as “pass” or “fail”.

Practical sessions: attendance (group assignment) and presentation (individual assignment): Participation in the practical sessions is compulsory. Every team must be present each week and present their progress and engage with other teams, and all team members are expected to be there. Every team member must present the team’s progress at least once. The assignment will be graded as "pass" or "fail".

Weekly progress report (individual assignment): We ask you to – individually – complete a simple weekly survey to indicate your team's progress, collaboration and communication and the participation of all team members (including yourself). The assignment will be graded as "pass" or "fail".

Compensation and final grade

Both the exam grade and the weighted average of the project assignment grades MUST be 5.5 or higher. Compensation between the two is not possible. Additionally, all "pass"/"fail"-assignments and presentations must be "pass" in order to pass the course.

- In case any of the pass/fail-assignments is a 'fail', the final grade of the course will be 'NVD', otherwise:
- In case any of the graded assignments is insufficient, the final grade of the course will be that insufficient grade, otherwise:
- In case all pass/fail-assignments are 'pass' and all graded assignments are at least 5.5, the final grade of the course will be the weighted average of the project assignment grade (75%) and the exam grade (25%).

Mode of re-examination

A failed project assignment can be revised and resubmitted once; a failed presentation can be retried once; failing or missing the game assignment CANNOT be retried. A failed exam can be retried in the resit period.

Literature

Len Bass et al., Software Architecture in Practice, 4th edition, 2021

Additional Information Target Audience

Master Information Sciences
Master Computer Science

Custom Course Registration

Group enrolment takes place in Canvas.

Explanation Canvas

Further information for this course will be made available online (Canvas). All students must be enrolled in the course Canvas community.

Recommended background knowledge

Software design.

Distributed Algorithms

Course Code	X_400211
Credits	6
Period	P5
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. W.J. Fokkink
Examiner	prof. dr. W.J. Fokkink
Teaching Staff	prof. dr. W.J. Fokkink, dr. Y. Nazari MSc
Teaching method(s)	Seminar, Lecture

Course Objective

The main objective is to develop an algorithmic frame of mind for solving fundamental problems in distributed computing, by obtaining insight into concurrency concepts. The course offers a bird's-eye view of a wide range of algorithms for basic and important challenges in distributed systems.

A key characteristic of the course is that correctness arguments and complexity calculations of distributed algorithms are provided in an intuitive fashion and by means of examples and exercises.

At the end of the course, the student will:

- be acquainted with fundamental aspects of distributed algorithms, notably logical clocks, wave algorithms, snapshots, rollback recovery, consensus, and self-stabilization; (knowledge and understanding)
- be able to apply algorithms for termination and deadlock detection, routing, leader election, and mutual exclusion; (applying knowledge and understanding)
- have a grasp of impossibility results regarding anonymous networks and networks in which nodes can crash; (knowledge and understanding)
- be aware of certain attacks, data structures (notably Merkle trees), and key exchange and authentication protocols to safeguard the security of distributed systems; (knowledge and understanding)
- have a clear understanding of advanced protocols for distributed transactions, cryptocurrencies, and quantum cryptography. (knowledge and understanding)

Course Content

The following topics are treated in the course: Logical clocks, snapshots, graph traversal, termination detection, garbage collection, deadlock detection, routing, election, minimal spanning trees, anonymous networks, checkpointing, fault tolerance, failure detection, consensus, mutual exclusion, self-stabilization, distributed

transactions, dynamic networks, key exchange, quantum cryptography, authentication, digital signatures, blockchains.

Additional Information Teaching Methods

Lectures: 4 hours per week. Exercise classes: 2 hours per week.

Method of Assessment

Written exam. At the exam, a physical copy or print-out of the textbook can be used. All books will be checked by the lecturer at the exam.

Literature

W.J. Fokkink, Distributed Algorithms: An Intuitive Approach (forthcoming 3rd edition, draft), MIT Press, 2025.

A pdf of the textbook is made available on the Canvas page.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Econometrics and Operations Research

Recommended background knowledge

Data Structures & Algorithms.

Distributed Systems

Course Code	X_400130
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	ir. J.J.R. Donkervliet MSc
Examiner	prof. dr. ir. A. Iosup
Teaching Staff	ir. S. Talluri, prof. dr. ir. A. Iosup, dr. T. De Matteis, ir. J.J.R. Donkervliet MSc
Teaching method(s)	Seminar, Lecture

Course Objective

By taking this course, you will learn to:

1. Explain the basic concepts, objectives, and functions of distributed computing (eco)systems, e.g., communication, resource management and scheduling, data consistency, fault-tolerance, and performance. (Knowledge and understanding)
2. Compare the basic characteristics of distributed computing with other computing paradigms (i.e., centralized, parallel). (Applying knowledge and understanding) (Making judgements)
3. Identify proficiently the different flavors of modern distributed ecosystems (i.e., peer-to-peer systems, cluster computing, grid computing, cloud computing, datacenters, distributed HPC, SDN, Big Data systems, IoT systems). (Knowledge and understanding) (Applying knowledge and understanding) (Making judgements)
4. Analyze proficiently the trade-offs inherent in the design of modern distributed systems. (Applying knowledge and understanding) (Making judgements)
5. Design your portfolio distributed system, with many basic and some complex operations of modern distributed systems. (Applying knowledge and understanding) (Making judgements)
6. Implement and deploy your portfolio distributed-system. (Applying knowledge and understanding) (Making judgements)
7. Analyze your portfolio distributed-system. (Communication) (Lifelong learning skills)

Course Content

This course focuses on distributed computing systems and ecosystems. In general, debugging and tuning existing systems and ecosystems, and designing, implementing, and analyzing new distributed computing systems remain

vital and challenging for both industry and academia.

Starting with the mid-1990s, computing is undergoing a revolution, in which collections of independent computers appear to users as a single, albeit distributed, computing system. Motivated by the advent of the Internet, the increase in the computation capacity of consumer computers, the commoditization of server-grade machines, energy constraints, etc., the distributed computing paradigm has permeated all fields using computers. Current distributed computing applications range from social networks to banking, from peer-to-peer file-sharing to high-performance computing used in research, from massively multiplayer online games to business-critical workloads, etc. Important advances have helped to fuse heterogeneous resources into truly global distributed systems and ecosystems, for example in scientific computing, where distributed computation is using Big Data and distributed sensors to produce meaningful progress for humankind. We will focus in this course on a number of these modern examples of distributed computing systems.

Although so many distributed systems and ecosystems already exist, the list of conceptual and technical challenges they pose is long. Depending on requirements, even trivial communication between nodes of the distributed system can be challenging. The failure of a single node, or sometimes even a performance hiccup, can bring an entire system down with it, other nodes or entire other systems may also crash, experiencing correlated and catastrophic failures. Data consistency and coordinating nodes remain important challenges made worse by the large scale of real-world deployments. Poor resource management and naive scheduling can lead to orders-of-magnitude higher operational costs and consumption of energy that we simply cannot spare. It is not uncommon for a modern distributed system to quickly rise and then fall in popularity, as exemplified by the 2016 example of Pokemon Go. We will present in this course real-world situations where modern distributed systems have behaved poorly.

Addressing these challenges requires unique approaches and concepts.

Separating concerns and breaking down problems into smaller cases often lead to limited success, because many properties of distributed systems can only be achieved end-to-end. Can anyone imagine a perfectly reliable production pipeline, if even one of its key stages can suffer failures?

Building capability by adding resources is often offset by the distributed nature of the system. Can anyone ignore the physical limitations of communication around the globe? In this course, we will focus on the unique approaches and principles of distributed systems, from specific architectures and communication protocols to specific concepts in resource management and scheduling, data consistency, fault-tolerance, and performance.

Additional Information Teaching Methods

The course uses prof. Iosup's gamification method. The course is taught as a series of lectures, in combination with self-study and design sessions, and with a large practical assignment. The course also has important design and seminar components.

Method of Assessment

Written exam. Report on the practical assignment. In-class assignments include: design, seminar, Q&A. There are other gamification elements.

For a total maximum of 10,000 points (equivalent to a grade of 10), the course activities will be graded as follows:

1. Exam, questions (max 6,000 points)
2. Practical, lab exercise (max 4,000 points)
3. Design and seminar sessions (max 2,000 points)
4. In-class activity: max 50p per session

Other bonuses may be applicable

There is only a resit opportunity for the exam.

Literature

The course uses the following textbook: Maarten van Steen and Andrew S. Tanenbaum, Distributed Systems, 3rd. Ed., online edition, 2017. (free for all) [Online] Available: https://www.distributed-systems.net/index.php/books/distributed-systems-

Recent material is only available as lecture slides and selected publications. The lecture slides also recommend additional literature.

The study guide available on Canvas also indicates other worthwhile sources of information.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Additional Information

This course uses the gamification technique developed by prof. Iosup and his team.

Explanation Canvas

The course guide is mandatory reading for every student prior to any other activity in the course.

Recommended background knowledge

Ability to work in teams for the practical assignment. Ability to design software and to develop code using modern software engineering practices, e.g., setting up your own GitHub repository, co-editing using tools such as Overleaf or Sharelatex, etc., is a big plus. Experience with big data processing, multimedia and online gaming systems, scientific (workflow) computing is also a plus.

Students should have taken standard courses on:

- Computer networks
- Operating systems
- Programming paradigms, in particular OOP and/or actor-based approaches
- Software engineering

Prior experience with Internet, web, distributed, or parallel programming and algorithms is helpful.

Prior experience with operating systems development and analysis, and in general experience with computer systems courses, is a big plus.

Dynamic Programming and Reinforcement Learning

Course Code	XM_0093
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. G.M. Koole
Examiner	prof. dr. G.M. Koole
Teaching Staff	V.G.M. Francois Lavet, prof. dr. G.M. Koole
Teaching method(s)	Seminar, Lecture

Course Objective

Dynamic Programming (DP) and Reinforcement Learning (RL) are fields concerned with decision making over time. After completing this course, the student

- is familiar with the commonly used algorithms for solving dynamic optimization problems;
- understands the main features of these algorithms, their strengths and weaknesses including their convergence properties;
- can implement them in an appropriate language;
- can model real-world decision problems into a DP or RL framework and solve moderately sized problems;
- has knowledge of the historical development of DP and RL and has an idea of possible future developments.

Course Content

This course is concerned with reinforcement learning and its origin dynamic programming. These are fields dealing with goal-directed decision making over time, such as finding your way in an unknown area, playing a game or pricing airline tickets.
We look at these areas from different angles:

- we deal with full-information "planning" problems, but also with partial-information "learning" problems
- we consider different algorithms, some of which are guaranteed to find the best solution, but also heuristics
- we consider high-dimensional problems (such as games) and methods to solve them

- we look at small toy problems to understand algorithms and sharpen our intuition, but also bigger problems for which we learn how to implement algorithms (in python)
- we look at different types of applications, both from AI (search problems, games) and OR

Additional Information Teaching Methods

Lectures and practical work integrated

Method of Assessment

Programming exercises and final exam.
The 3 assignments each count for 10% and the exam for 70%.
The minimal passing grade for the exam is 5.0.

Literature

Slides and lecture notes

Additional Information Target Audience

mBA, mAI, mCS, mBa-D, mMath

Recommended background knowledge

Programming experience in python

Evolutionary Computing

Course Code	X_400111
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. A.E. Eiben
Examiner	prof. dr. A.E. Eiben
Teaching Staff	prof. dr. A.E. Eiben, mr. J.M. di Matteo, dr. K. da Silva Miras de Araujo
Teaching method(s)	Seminar, Lecture

Course Objective

This course has a threefold objective:

1. (Knowledge & Understanding) To learn about computational methods based on Darwinian principles of evolution.
2. (Applying knowledge and understanding, Communication) To use such methods as problem solvers and as simulation tools; to determine adequate algorithm setups depending on the problem at hand.
3. (Applying knowledge and understanding, Lifelong learning skills) To gain hands-on experience in performing computational experiments with evolutionary algorithms.

Course Content

This course is about constructing, applying and studying algorithms based on the Darwinian evolution theory. Driven by selection (survival of the fittest, mating of the fittest) and randomised reproduction (mutation, recombination), an evolutionary process is being emulated and solutions for a given problem are being "bred". During this course, various flavours within evolutionary computing are treated, including genetic algorithms, evolution strategies, evolutionary programming, genetic programming, differential evolution, particle swarm optimisation. Applications in optimisation, constraint handling, machine learning, and robotics are discussed. Specific subjects handled include: genetic structures (representations), selection techniques, sexual and asexual reproduction operators, (self-)adaptivity and methodological aspects, such as algorithm design & tuning, and performance assessment. Special attention is paid to the field of evolutionary robotics where the bodies (morphology, hardware) and the brains (controller, software) of robots are evolved simultaneously. Hands-on-experience with evolutionary algorithms is gained through a compulsory programming assignment.

Additional Information Teaching Methods

Oral lectures and compulsory Python programming assignment (in teams of 3 to 5). A limited number of highly motivated students can replace the programming assignment by a special research assignment under the personal supervision of the lecturer(s). These research projects aim at publications with the students as first authors of the paper.

Method of Assessment

Written exam and programming assignment (in teams of 3 to 5). To pass the course as a whole, you must pass both the exam and the programming assignment. For the final grade they will count with the same weight:

Grade = 0.5 x Exam_Grade + 0.5 x Assignment_Grade

Note that:

- The exam is multiple choice, taken on the campus in an electronic form. There is a resit in case you cannot attend or fail.
- The programming assignment is graded based on your code, the quality of the solution your algorithm finds, and a short report. **No resit is possible for the programming assignment.**
- If you pass one component (Exam or Assignment), but fail the other, then you can keep the grade of the passed component for next year or later. Then you can pass the course by getting a good grade (> 5.5) for the missing component and using the old grade in the averaging formula.

Literature

The course is based on the textbook ***A.E. Eiben and J.E. Smith, Introduction to Evolutionary Computing, Springer, 2015, 2nd edition, ISBN 978-3-662-44873-1***. The lectures and the book chapters are synchronised to a great extent. In addition, a number of carefully selected papers to provide extra information on specific topics are available via Canvas.

Additional Information Target Audience

MSc Artificial Intelligence
MSc Econometrics and Operations Research
MSc Finance
MSc Business Analytics
MSc Computer Science
MSc Computer Security
MSc Bioinformatics and Systems Biology

Additional Information

Attending the lectures in real life is not mandatory, but highly recommended. Active participation in the team work for the programming assignment is a must!

Explanation Canvas

The complete schedule of the course, organised by the chapters of the textbook, will be made available at the beginning of the course. Students are assumed to consult the course schedule to keep up with the progress, as Canvas rubrics are not used. Video recordings of the lectures made during the COVID period are still available on Canvas for self-study.

Recommended background knowledge

Python programming skills are necessary to do the practical assignment. If you cannot program, you cannot complete this course.

Experimental Design and Data Analysis

Course Code	X_405078
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. E.N. Belitser

Examiner	dr. E.N. Belitser
Teaching Staff	dr. E.N. Belitser
Teaching method(s)	Practical, Lecture

Course Objective

In this course, students will become familiar with the most commonly used experimental designs, regression models, some nonparametric tests and bootstrap methods. By the end of the course, students will be able to:

- Identify and describe some of the most widely used statistical models,
- Understand the basic theory underlying these models,
- Apply these models appropriately in relevant contexts.

Course Content

The EDDA course introduces a range of widely used statistical models and techniques, including paired and independent t-tests, non-parametric rank-based tests, bootstrap and permutation methods, analysis of variance (ANOVA), ANCOVA, lasso, linear regression models, generalized linear and nonlinear models, and survival analysis. Students will learn how to choose appropriate models for various scenarios, estimate model parameters using available data, and validate model fit. For regression models, proper study design is essential to draw sound conclusions; the course covers well-known experimental designs such as completely randomized and randomized block designs, along with their associated ANOVA techniques. A portion of the course focuses on non-parametric and resampling methods, such as the Wilcoxon test (one- and two-sample), Kolmogorov–Smirnov test, rank correlation tests, and both permutation and bootstrap tests. The methods will be illustrated using practical examples, with an emphasis on real datasets. The statistical software R will be used for data analysis throughout the course.

Additional Information Teaching Methods

Lectures, practical sessions.

Method of Assessment

Course assessment consists of assignments (33.3%) carried out in teamwork during the course and an individual digital test (66.7%) at the end. To pass the course, both components must meet minimum required thresholds. Re-examination policy: if the final result is insufficient, only the individual test may be retaken.

Literature

Literature will be communicated on Canvas.

Additional Information Target Audience

MSc AI, CS

Additional Information

Assignments are to be made using the statistical package R (http://www.r-project.org).

Explanation Canvas

All the information about the course will be available on Canvas.

Recommended background knowledge

- A basic course in statistics of the same level as Statistical Methods (X_401020) or Statistical Methods for AI (XB_0080) is mandatory. All the material and its prerequisites (basic mathematical and programming skills) for these courses will be assumed known in the lectures/assignments/exam of the course EDDA.
- A good background in probability, at least of the same level as in Statistical Methods (X_401020) or Statistical Methods for AI (XB_0080).
- Basic knowledge of the statistical software R and its application to data analysis.

Foundations of Cryptography

Course Code	XM_0174

Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	S.A. Oechsner
Examiner	S.A. Oechsner
Teaching Staff	S.A. Oechsner
Teaching method(s)	Seminar, Lecture

Course Objective

Upon completion of this course:

- Students are familiar with the most fundamental cryptographic building blocks and the security guarantees they can(not) provide. (Knowledge and understanding)
- Students know the basic security models used in cryptography. (Knowledge and understanding)
- Students can reason about the security of basic cryptographic building blocks in these models. (Applying knowledge and understanding)

Course Content

Cryptography forms the foundation for the security of many digital systems around us. In this course, we will study how the basic cryptographic building blocks work, and why they work: What does it mean to be secure? How can security be formally defined, and how does one formally reason about security?

Covered topics include:

- the provable security mindset
- pseudorandom functions
- symmetric encryption
- message authentication codes
- collision-resistant hash functions
- public-key encryption
- digital signatures

Additional Information Teaching Methods

Lectures (4 hours per week) and exercise class (2 hours per week).

Method of Assessment

Written exam.

Literature

We will be using "The Joy of Cryptography" by Mike Rosulek as textbook. Available as free pdf at https://joyofcryptography.com.

Recommended background knowledge

Knowledge of basic discrete mathematics (simple combinatorics, discrete probability, modular arithmetic). Prior exposure to abstraction in the context of computer science is helpful, e.g. in the form of automata theory as taught at the basic CS bachelor level.

Green Lab

Course Code	X_418158
Credits	6
Period	P1
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. I. Malavolta

Examiner	dr. I. Malavolta
Teaching Staff	dr. I. Malavolta, prof. dr. P. Lago, dr. V. Stoico
Teaching method(s)	Practical, Lecture

Course Objective

Upon completion of this course, students will:

- Learn the principles of Empirical Software Engineering. (Knowledge and understanding)
- Be able to operate in a lab environment and build a successful experiment for software energy consumption. (Applying knowledge and understanding) (Making judgements) (Lifelong learning skills)
- Become familiar with relevant research problems in the field of Green Software Engineering. (Knowledge and understanding)
- Understand and measure the impact of software over energy consumption. (Applying knowledge and understanding) (Making judgements)

Course Content

Students will work in teams to perform experiments on software energy consumption in a controlled environment. Students will have to carry out all the phases of empirical experimentation, from experiment design to execution, data analysis, and reporting. Students will be provided with examples of experiments coming from state-of-the-art literature, but they will have to choose by themselves the experimental subjects and hypotheses to test. During the lab sessions, students will be assisted for technical operation of the lab equipment as regards measurement and data gathering. Students will also receive the required training for data analysis and visualization using specialized software (e.g., R and RStudio).

Additional Information Teaching Methods

Lectures (H). Lab sessions (pra).

Method of Assessment

Students will be assessed via (1) a team project and (2) an individual presentation.

The team project will be carried out throughout the whole course by groups of 5 students; each part of the project will be discussed during the lectures, started during the labs, and completed as homework so that students will likely be on track within the course schedule.

The result of the team project is about planning, designing, conducting, and reporting a successful experiment for software energy efficiency in a specific technological context, such as mobile applications, robotics, and microservices. In this way, students will put into practice the skills and techniques that they have learned during the lectures and develop their practical insights by applying them to real software. The experiment will be evaluated by the instructor starting from a written report describing the main aspects of the experiment and according to a shared assessment rubric (presented in the first lecture of the course).

Presentations will be prepared as group work and they will be about the main points of the team project. Each student will present individually a part of the presentation, which will be graded as either “pass” or “fail”.

All assessment components (e.g., team project and individual presentations) have a resit opportunity.

Literature

- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. Experimentation in software engineering. Springer, 2012.
- I. Malavolta, V. Stoico, P. Lago. Ten Years of Teaching Empirical Software Engineering in the context of Energy-efficient Software, 2024. Online: https://arxiv.org/abs/2407.05689
- Material distributed on Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Recommended background knowledge

Python programming and shell scripting.
Basic statistical analysis techniques (descriptive statistics and most common tests).
Academic writing.

Hardware Security

Course Code	XM_40019
Credits	6
Period	P2
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. C. Giuffrida
Examiner	prof. dr. C. Giuffrida
Teaching Staff	prof. dr. C. Giuffrida
Teaching method(s)	Lecture

Course Objective

This course covers the security of low-level computer hardware, ranging from the memory subsystem on modern CPUs to deeply embedded systems present almost everywhere in our everyday life. The aim of the course is for you to familiarize with the emerging field of hardware security, more specifically micro-architectural and circuit-level attacks, side-channel analysis, firmware analysis, etc. Furthermore, you will learn about the current state of the art by reproducing the results of cutting-edge research and gathering hands-on practical experience.

Be prepared to build some of the most advanced exploits on the planet! This is the course where you get credit points by exploiting the CPU on your computer, breaking into embedded systems, and exfiltrating bits directly from the circuit board!

The luckiest team will collect the infamous Best Demo Award given each year at the end of the course.

Upon completion of this course, you will:

- have knowledge about the different offensive and defensive research areas involving low-level hardware security (Knowledge & Understanding)
- have sufficient knowledge and experience to explore open research problems in at least one area of hardware security (Lifelong learning skills)
- be able to reproduce micro-architectural attacks from recent research (Knowledge & Understanding, Applying Knowledge and Understanding, Making Judgements, Communication)
- be familiar with techniques used to perform firmware analysis (Knowledge & Understanding, Applying Knowledge and Understanding, Making Judgements, Communication)
- have knowledge of common hardware communication interfaces and their use in typical hardware. (Knowledge & Understanding, Applying Knowledge & Understanding)

Course Content

The course is divided in two parts. In the first few weeks, you will learn the basics of hardware security with different topics (e.g., reverse engineering, side-channel analysis, classical attacks, firmware analysis, etc.) covered in lectures and individual practical assignments. The lectures and the individual assignments run in parallel and are meant to prepare you for a final group project in the second part of the course. In your group project, you will get the opportunity to dive into one of the topics covered in the course in depth, by replicating or even improving on cutting-edge hardware security research.

Additional Information Teaching Methods

Lectures and practical assignments.

Method of Assessment

The final grade is determined by (i) the quality and correctness of solutions to the individual assignments and answers based on oral discussions about them (50%), as well as (ii) the quality of the solution of the group project (50%). Each component must be passed with a grade >= 4. The final grade is the weighted average of the components' grades.

Please note that your grade will be registered as follows:

- If you do not submit any assignment and do not participate in any oral discussion, you will receive a NS.
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments, your grade is NVD.

- Otherwise, your final grade is your weighted average.
- If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

There is no resit opportunity for the practical assignments, nor for the oral discussions. Results for practical assignments and for the oral discussion may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner). If you submit an assignment or participate in the oral discussion, the new result replaces any transferred results.

We ensure that each student individually demonstrates their abilities through oral exams to test understanding of the assignments, through active plagiarism detection for the assignments, and by investigating whether each group member contributed sufficiently for the group assignment.

Literature

No set book. All material will be made available during the course.

Additional Information Target Audience

MSc Computer Security

Recommended background knowledge

Knowledge of operating systems, C, computer architecture, and a keen interest in understanding how things work under the hood. Having attended the Binary and Malware Analysis course is strongly recommended. Having attended the Advanced Operating Systems course is a plus

High Performance Computing and Big Data

Course Code	XMU_40013
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130398

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Introduction to Computational Science

Course Code	XMU_418111
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130584

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Large Research Project in Computer Security

Course Code	XM_0126
Credits	12
Period	Ac. Year (September)
Course Level	600
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. C. Giuffrida
Examiner	prof. dr. C. Giuffrida
Teaching Staff	prof. dr. C. Giuffrida
Teaching method(s)	

Course Objective

The objectives of the course are:

- [Applying Knowledge and Understanding] After completing the course, the student will be able to execute a computer security project in the context of ongoing research.
- [Making judgements] The student will be able to evaluate their own approach and results w.r.t. novelty and efficacy.
- [Communication] The student will be able to communicate their results and findings in a scientific report.
- [Lifelong learning skills] The student will be trained in applying their existing knowledge and skills in the context of ongoing research.

Course Content

The aim of the course is to give very motivated students the opportunity to work on a challenging, research-oriented project, with a strong focus on strong computer security conceptual and/or practical work.

The content of the project is to be negotiated individually with a VU Computer Science lecturer, who will supervise and grade it. This must be done before the project starts. In principle, only a few students (the ones that submit a convincing project proposal) will be allowed to take this course. The project can be carried out as a ‘duo project’, in which case the proposal should indicate the individual contribution of each student.

The project is supposed to apply state-of-the-art concepts, methodologies and/or technology and requires an original contribution by the student/s. It must be challenging, and allow a clear assessment of the student's own contribution.

At the end of the project, the student submits a written report to the supervisor, describing the work, the lessons learned, and the insights gained from applying study contents in a research-oriented computer security project.

The report contains a description of the project, including the problem description, the student’s solution, and an evaluation of the results. The report also contains a reflection on the novelty of the achieved research results, a comparison with related work, and what the student has learned during the project. In case of a duo project, the report should clearly explain the individual contribution of each student so as to personalize their contribution to the relevant parts.

Additional Information Teaching Methods

Guided project work.

Method of Assessment

The overall grade is based on two components: research work (70%) and the quality of the written report (30%). Each component must be passed with a grade >= 5.5. The final grade is the weighted average of the components’ grades. The report will be evaluated by the supervisor and by a second reader. For grading, the supervisor should complete and submit a special evaluation form on Canvas (uploaded on the Master Computer Security Canvas page).

We ensure that each student individually demonstrates their abilities through close supervision of the research/writing work.

Literature

To be distributed individually, by the project supervisor.

Additional Information Target Audience

MSc Computer Security

Additional Information

The Research Project in Computer Security and the Large Research Project in Computer Security are identical, except for the size (and EC) of the student project. Students can follow either one, or both.

Custom Course Registration

For being accepted, students must contact their prospective supervisor directly.

Recommended background knowledge

The student should have completed at least 42 credits of their Master program, so that there is sufficient study content to be applied in the project.

Machine Learning for the Quantified Self

Course Code	XM_40012
Credits	6
Period	P6
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science

Course Coordinator	prof. dr. M. Hoogendoorn
Examiner	prof. dr. M. Hoogendoorn
Teaching Staff	V.G.M. Francois Lavet, prof. dr. M. Hoogendoorn, B. Borsboom
Teaching method(s)	Practical, Lecture

Course Objective

The main aims of this course are to make the student familiar with specific machine learning techniques for quantified self and sensory data. Both the theory as well as the practical side of matters will be addressed. More specifically, the following aims are distinguished:

1. Understanding the domain of the quantified self and sensory data
2. Knowing and being able to apply techniques for outlier detection on sensory data.
3. Knowing and being able to apply techniques for feature engineering on sensory data.
4. Knowing and being able to apply clustering techniques related to sensory data.
5. Understanding the theory of supervised learning and its implications.
6. Learning how to apply non-temporal machine learning techniques to sensory data.
7. Knowing and being able to apply temporal machine learning techniques to sensory data, including deep learning techniques.
8. Knowing and being able to apply reinforcement learning in the application of the quantified self.

The course if focused on the Dublin descriptors Knowledge and understanding (learning new relevant algorithm in this domain); Applying Knowledge and Understanding (since the aim is also to apply the algorithms in practice); Making judgements (which technique is appropriate, how to best apply the techniques for a specific case); Communication skills (how to report on your approach, choices and your results), and Learning skills (able to find new relevant techniques, assess their suitability, etc.).

Course Content

The quantified-self refers to large-scale data collection of a user's behavior and context via a range of sensory devices, including smart phones, smart watches, ambient sensors, etc. These measurements contain a wealth of information that can be extracted by means of machine learning techniques, for instance for the purpose of predictive modeling. In addition, machine learning techniques can be a driver for adaptive systems to support users in a personalized way based on the aforementioned measurements. The type of data does however require specialized machine learning techniques to fully exploit the information contained in the data. Examples of challenges include the temporal nature of the data, the variety in the type of data, the different granularity of various sensors, noise, etcetera.

In this course specific techniques to handle quantified self (or broader sensory data) will be treated. More in specific, it will address:

- Feature engineering (how do we come from raw data to usable features):
  * Removing noise from data
  * Handling missing data
  * Identifying (temporal) features
- Learning of user patterns:
  * Temporal machine learning approaches such as recurrent neural networks, time series analysis
  * Clustering approaches with dedicated distance metrics (including dynamic time warping)
- Adaptive feedback and support
  * Reinforcement learning
- Integration of the various components.

In addition, a number of real-life applications will be discussed. Next to lectures, there will be an extensive practical part, where students will learn to work with various algorithms work on a project they propose themselves.

Additional Information Teaching Methods

The course will be taught in four weeks. Both regular lectures (l, first two weeks) and practical sessions (p) will be organized. During the first two weeks the emphasis will be on the lectures and getting started with the practical assignment that ties closely to the content of the lectures. The third week will be working on the practical assignment fulltime. In the fourth week a mandatory final lecture will take place in which a selection of groups will present their project. The course ends with an exam.

Method of Assessment

Written exam (50%) and practical assignment (50%). For both parts, the grade needs to be sufficient to obtain a final grade. The practical assignment is done in groups of three. For the practical assignment, two intermediate versions of the report should be submitted for feedback that are given a pass/fail. Only in case of a pass for the two

intermediate versions will a final grade be given for the practical assignment.

The written exam will be multiple choice and have a resit during the Summer period. On top, in case your assignment grade average is insufficient you are allowed to redo the assignments by improving it based on the feedback you received. The maximum grade for this resubmitted assignment is set to a 6.

Literature

Hoogendoorn, M. and Funk, B. Machine Learning for the Quantified Self - On the Art of Learning from Sensory Data, ISBN 978-3-319-66308-1, Springer Verlag 2018. The book can be downloaded free of charge through the UBVU, just follow the link to the book: http://dx.doi.org/10.1007/978-3-319-66308-1

Papers on state-of-the-art deep learning techniques.

Additional Information Target Audience

Master Artificial Intelligence, Master Business Analytics, Master Computer Science

Additional Information

Lecturer: Prof.dr. M. Hoogendoorn

Custom Course Registration

Registration for medical informatics (UvA) students: Please check the following document https://www.dropbox.com/s/a2kd7xacuxt5y8e/AI%20for%20Health%202022-2023.
and then register for secondary courses via https://vu.nl/en/student/information-about-registration-and-enrolment/in

Recommended background knowledge

Programming experience. Knowledge of basic machine learning algorithms.

Master Project Computer Security

Course Code	XM_0123
Credits	30
Period	Ac. Year (September)
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	mr. S. Ud Din
Examiner	mr. S. Ud Din
Teaching Staff	mr. S. Ud Din
Teaching method(s)	Lecture

Course Objective

By completing this project, a student will:

1. have solid scientific knowledge and understanding of computer security with a “systems” emphasis, including knowledge of programming, computer systems, software security, hardware security, and program analysis (Knowledge and understanding)
2. have acquired practical skills in relevant areas of computer security at an academic level (Knowledge and understanding)
3. be aware of the applications of computer science in general and of computer security in particular, and be capable of applying their knowledge and skills to new and unfamiliar problems (Applying knowledge and understanding)
4. be capable of performing effectively in professional situations where scientific knowledge and skills in the field of computer security are required (Making judgements)
5. be capable of communicating at a professional level and of giving clear oral and written presentations of their own work or others’ work (Communication)
6. have developed a broad set of lifelong learning skills in computer security, including being capable of acquiring new knowledge, designing a research plan/project, conducting independent/collaborative research, consulting international professional literature, and critically analyzing/evaluating scientific results (Lifelong learning skills)

Course Content

The student is offered a research question that needs to be answered following a systematic approach. This approach includes steps such as exploring relevant literature, and will, in general, consist of setting up and carrying out experiments by means of simulations, emulations, or actual systems software. The results and findings will be described in a thesis conforming to the academic standards in the field. The thesis must contain an appendix documenting any research artifacts produced as part of the implementation and experimental evaluation. The appendix should use the provided Artifact Evaluation template (part of the official Latex thesis template uploaded on the Master Computer Security Canvas page). An oral presentation of the project results concludes the project.

Additional Information Teaching Methods

The Master Project is supervised by a staff member of the Department of Computer Science of the VU.

Method of Assessment

The final grade is determined by the project supervisor and a second reader, based on three components: the quality of the research (3/6), the written thesis (2/6), and an oral presentation (1/6). Each component must be passed with a grade >= 5.5. The final grade is the weighted average of the components' grades. There is a resit opportunity for an insufficient presentation.

We ensure that each student individually demonstrates their abilities through close supervision of the research and writing, through an individual presentation, and through active plagiarism detection for the thesis.

Entry Requirements

The student must have accumulated a minimum of 84 EC in the study program before starting the Master Project.

Additional Information Target Audience

MSc Computer Security

Network Security

Course Code	XM_0100
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	B. Chandrasekaran
Examiner	B. Chandrasekaran
Teaching Staff	B. Chandrasekaran, dr. M. Zhang
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

- Understand how the implicit notion of trust in the design of Internet protocols introduces numerous vulnerabilities across different protocols (K&U, LS).
- Understand how the web public-key infrastructure (PKI) ecosystem works, focussing both on the theory and (mal)practice of various security mechanisms and protocols (K&U, AK&U, LS, C).
- Analyze how attackers abuse the vulnerabilities (e.g., DNS spoofing, BGP route hijacks, and various amplification attacks), by studying several recent attacks (e.g., using DNS, BGP, and TCP protocols) (K&U, AK&U, MJ, C).
- Study the state-of-the-art mechanisms for mitigating today's attacks and review their effectiveness (K&U, AK&U, C).
- Learn how to inspect the design and development, and investigate the deployment of various security mechanisms or protocols (K&U, AK&U, LS).

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This course focuses broadly on network and (the network side of) web security. We will focus on analyzing vulnerabilities in Internet protocols (e.g., DNS, BGP, and TCP), attacks (e.g., sniffing, spoofing, and hijacking) that exploit such vulnerabilities, and mechanisms to mitigate such attacks.

To learn the “state-of-the-art” attacks and mitigation strategies, the course will review and discuss several scientific papers published at top-tier venues.

Additional Information Teaching Methods

Lectures, quizzes, and mid-term and final exams.

Method of Assessment

Quizzes (30%), mid-term exam (30%), and final exam (40%).

To pass the course, you must satisfy the following criteria: (a) You must score at least 6.0 on average in the quizzes; (b) you must score at least 6.0 in the final exam, and (c) You must score 6.0 in total (i.e., with all three components taken together).

There is no resit opportunity for the quizzes or the mid-term exam. There will be one resit opportunity for the final exam.

This course does not allow students to carry over (partial) grades from last year.

[Fraud] We ensure that each student individually demonstrates their abilities through the requirement that the supervised mid-term and final exam be passed with an average of at least 6.0.

Please note that your grade will be registered as follows:
If you do not submit any review and do not present a paper, you will receive a NS.
In all other cases, you will receive either a numeric grade or incomplete (NVD).
If you did not meet the minimum grade requirements for the reviews, your grade will be NVD.
Otherwise, your final grade is your weighted average.
If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

Literature

No set book. All materials will be made available during the course.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

A basic knowledge of network protocols, and a good grasp of probability as well as simple data structures and algorithms.

Parallel Programming Practical

Course Code	X_400162
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. Verdugo Rojano
Examiner	dr. F. Verdugo Rojano
Teaching Staff	mr. M. Cassar, drs. C. Verstoep, dr. F. Verdugo Rojano
Teaching method(s)	Lecture

Course Objective

At the end of the course, the students will:

- Be able to apply distributed parallel programming models to implement algorithms on large-scale computing systems. (Applying knowledge and understanding)
- Have hands-on experience in executing and testing parallel programs using a cluster computing environment. (Applying knowledge and understanding, making judgments)
- Be able to critically evaluate the performance of a distributed parallel program, identifying bottlenecks, and perform optimizations. (Applying knowledge and understanding, making judgments)
- Effectively communicate parallelization approaches and performance results by writing scientific reports. (Communication, lifelong learning skills)

Course Content

This course is a follow-up of "Programming Large-Scale Parallel Systems", which allows students to apply the knowledge studied in "Programming Large-Scale Parallel Systems" in a more practical, hands-on setting. Throughout the course, students will transform given sequential algorithms into parallel programs able to run on multiple compute nodes within a large-scale computing system. We will consider distributed programming models, including message-passing with MPI and the task-oriented model based on remote procedure calls included in the Julia programming language. The programs will be tested for correctness and benchmarked for performance on the DAS-5 cluster computer (see http://www.cs.vu.nl/das5). The performance of the program will be measured, analyzed and, whenever necessary, optimized. Profiling tools will be considered to identify bottlenecks and to guide optimization. A scientific report must be written for each problem discussing the parallelization approach, the implementation choices, and the outcomes of the performance analysis.

Additional Information Teaching Methods

- A kick-off meeting to introduce the course objectives and the practical assignments.
- After the kick-off meeting, students work independently towards the completion of the assignments.
- There are weekly meetings for Q&A and to guide students in different aspects of the work.

Method of Assessment

- (90%) Practical assignments that include practical programming work and written report.
- (10%) Individual oral exam, where students defend their work in a presentation about the assignments and by answering questions from the teachers.

Additional rules

- The grade of the practical assignments should be at least 5.5 to pass the course.
- The grade of the individual oral exam should be at least 5.5 to pass the course.
- There is no resit opportunity for the practical assignments nor the oral exam.

Literature

All course materials, including documentation and reference materials, will be made available through Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Computational Science

Recommended background knowledge

- We highly recommend having attended "Programming Large-Scale Parallel Systems" before joining "Parallel Programming Practical".
- The algorithms will be implemented in the Julia programming language. An introduction to parallel computing in Julia is given in the course "Programming Large-Scale Parallel Systems". Otherwise, students can study the lecture notes in https://www.francescverdugo.com/XM_40017/dev/.

Performance of Networked Systems

Course Code	X_405105
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English

Faculty	Faculty of Science
Course Coordinator	prof. dr. R.D. van der Mei
Examiner	prof. dr. R.D. van der Mei
Teaching Staff	prof. dr. R.D. van der Mei
Teaching method(s)	Lecture

Course Objective

Students will acquire basic knowledge of:

- Performance issues in communication systems, a complex interplay between communication networks (TCP/IP, UDP), application servers (HTTP, QUIC), services and clients
- Performance models for wireless and mobile network protocols (WiFi, GSM, GPRS, UMTS, HSDPA, 4G, 5G, 6G and mobile ad-hoc networks)
- Performance engineering principles and management of traffic in IP-based networks
- Performance modeling of streaming and elastic traffic
- Quantitative models for planning capacity in communication networks

Students will gain experience in modelling, engineering and planning performance of networked systems, and will learn how to tackle performance problems arising in today's information and communication systems.

Course Content

Over the past few decades, information and communication technology (ICT) has become ubiquitous and globally interconnected. As a consequence, our ICT systems are expected to process huge amounts of digital information, which puts a tremendous burden on our ICT infrastructure. At the same time, our modern society has become largely dependent on the well-functioning of our ICT systems: large-scale system failures and perceivable Quality of Service (QoS) degradation may completely disrupt our daily lives and have huge impact on our economy. These developments raise the need for understanding the nature and source of performance problems, and for finding effective ways to make our networks robust and reliable.

Motivated by this, the course will address performance-related challenges and solution methods in communication networks. To this end, we focus on performance modeling, traffic management, QoS provisioning and capacity planning for both wired and wireless/mobile communication networks (such as WiFi, 2G, 3G, 4G, 5G, 6G networks, and mobile ad-hoc networks). We address questions like:

- How do wired and wireless communication network technologies work?
- How can we model their performance?
- How can we manage and control traffic flows and provision QoS in IP-based networks?
- How can we deal with the omnipresence of uncertainty in virtually all aspects of communication networks?
- What techniques are available to model, analyze and optimize performance?

Additional Information Teaching Methods

Classroom lectures and a practical take-home homework assignment.

Method of Assessment

The assessment will be based on the homework assignment (total 20%) and a written exam (80%). Homework assignments can be made in groups of one or two students.

Literature

- A textbook on "High Performance Browser Networking" by Ilya Grigorik, O-Reilly, 2013.
- A reader on Stochastic Performance Modelling.
- Individual papers on specific topics.

Additional Information Target Audience

mBA, mCS, mPDCS, mCLS, mEctrie

Explanation Canvas

The relevant course information will be shared via CANVAS.

Recommended background knowledge

The students should have basic knowledge of computer networks.

Programming Large-Scale Parallel Systems

Course Code	XM_40017
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. Verdugo Rojano
Examiner	dr. F. Verdugo Rojano
Teaching Staff	dr. F. Verdugo Rojano, mr. M. Cassar
Teaching method(s)	Lecture

Course Objective

At the end of the course, students will be able to

- Understand the main principles of High Performance Computing (Knowledge and understanding)
- Design and implement efficient parallel algorithms on distributed-memory parallel computers (Applying knowledge and understanding)
- Make use of different distributed parallel programming models to implement these algorithms, including message passing with MPI and the task-based model based on remote procedure calls available in the Julia programming language (Applying knowledge and understanding).
- Analyze the performance of parallel applications and network topologies (Making judgements)
- Identify the main parallelization challenges in selected scientific computing applications (Knowledge and understanding)

Course Content

In this course, students learn how to design and implement computer programs that solve a problem in parallel on a distributed-memory computing system such as the supercomputers available in the top500 list (https://top500.org/). This is key in many real world applications to reduce execution time and to solve very large problems that cannot fit in a single compute node. You will learn how to design such parallel algorithms in order to minimize different types of performance overheads such as communication overhead, search overhead, and load imbalance. You will also learn how to analyze the theoretical and experimental performance of the resulting codes and the underlying computer network topology. Among all parallel programming models available for high performance computing, this course focuses on distributed-memory parallelization. We consider different distributed parallel programming models, including established techniques such as message-passing with MPI, but also modern approaches like the task-based model available in the Julia programming language. The course also discusses the application of these techniques to a set of real-world scientific computing applications, including partial-differential equations in solid and fluid dynamics, and n-body problems in astrophysics.

Additional Information Teaching Methods

- There are 2 class sessions per week. Part of the course follows the flipped classroom method. Pre-recordings, lecture notes, and exercises are made available in advance for self study, and follow up discussions take place in class meetings to deepen understanding.
- The course also includes the opportunity to run experiments on the DAS-5 computer cluster (http://www.cs.vu.nl/das5).
- Computer codes used in the course are implemented in the Julia programming language. Previous knowledge of Julia is not required as it will be learned as part of the course.

Method of Assessment

There is a final written exam (E) and a programming assignment (A). Their grades are combined as G = 0.1*A + W(E)*E, where

- W(E) = 1 for E < 7, and
- W(E) = 1-(E-7)/30 otherwise.

The grades E and A are at most 10 points each.

According to this formula the programming assignment is "quasi optional". There is little or no penalty for skipping

the assignment in most of the cases. However, high-scoring students in the exam need the assignment for a perfect score.

Literature

Course material will be made available on Canvas. Part of the course is based on the lecture notes available at https://www.francescverdugo.com/XM_40017/dev/.

Additional Information Target Audience

Master Business Analytics
Master Computational Science
Master Computer Science
Master Computer Security

Additional Information

There is also a separate course "Parallel Programming Practical" (6 EC) in Period 3, the aim of which is to complement the contents of this course with further practical skills and experience.

Research Project in Computer Security

Course Code	XM_0125
Credits	6
Period	Ac. Year (September)
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. C. Giuffrida
Examiner	prof. dr. C. Giuffrida
Teaching Staff	prof. dr. C. Giuffrida
Teaching method(s)	

Course Objective

The objectives of the course are:

- [Applying Knowledge and Understanding] After completing the course, the student will be able to execute a computer security project in the context of ongoing research.
- [Making judgements] The student will be able to evaluate their own approach and results w.r.t. novelty and efficacy.
- [Communication] The student will be able to communicate their results and findings in a scientific report.
- [Lifelong learning skills] The student will be trained in applying their existing knowledge and skills in the context of ongoing research.

Course Content

The aim of the course is to give very motivated students the opportunity to work on a challenging, research-oriented project, with a strong focus on strong computer security conceptual and/or practical work.

The content of the project is to be negotiated individually with a VU Computer Science lecturer, who will supervise and grade it. This must be done before the project starts. In principle, only a few students (the ones that submit a convincing project proposal) will be allowed to take this course. The project can be carried out as a ‘duo project’, in which case the proposal should indicate the individual contribution of each student.

The project is supposed to apply state-of-the-art concepts, methodologies and/or technology and requires an original contribution by the student/s. It must be challenging, and allow a clear assessment of the student's own contribution.

At the end of the project, the student submits a written report to the supervisor, describing the work, the lessons learned, and the insights gained from applying study contents in a research-oriented computer security project.

The report contains a description of the project, including the problem description, the student’s solution, and an evaluation of the results. The report also contains a reflection on the novelty of the achieved research results, a comparison with related work, and what the student has learned during the project. In case of a duo project, the

report should clearly explain the individual contribution of each student so as to personalize their contribution to the relevant parts.

Additional Information Teaching Methods

Guided project work.

Method of Assessment

The overall grade is based on two components: research work (70%) and the quality of the written report (30%). Each component must be passed with a grade >= 5.5. The final grade is the weighted average of the components' grades. The report will be evaluated by the supervisor and by a second reader. For grading, the supervisor should complete and submit a special evaluation form on Canvas (uploaded on the Master Computer Security Canvas page).

We ensure that each student individually demonstrates their abilities through close supervision of the research/writing work.

Literature

To be distributed individually, by the project supervisor.

Additional Information Target Audience

MSc Computer Security

Additional Information

The Research Project in Computer Security and the Large Research Project in Computer Security are identical, except for the size (and EC) of the student project. Students can follow either one, or both.

Custom Course Registration

For being accepted, students must contact their prospective supervisor directly.

Recommended background knowledge

The student should have completed at least 42 credits of their Master program, so that there is sufficient study content to be applied in the project.

Secure and Trustworthy Machine Learning

Course Code	XM_0179
Credits	6
Period	P3
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. ir. M.E. van Dijk
Examiner	prof. dr. ir. M.E. van Dijk
Teaching Staff	M. Chen, prof. dr. ir. M.E. van Dijk
Teaching method(s)	Study Group, Lecture

Course Objective

We will discuss various topics in the intersection of security and machine learning. The main goal of this course is to have you think critically about how and to what extent security guarantees can be given for machine learning tasks. The main objective is to study trustworthy intelligence augmentation from different perspectives.

Upon completion of this course, you will:

- Have knowledge about security problems that arise in machine learning (K&U, AK&U, C)
- Understand (mathematical) technique(s) towards remediating security problems (K&U, AK&U, C)
- Have knowledge of example(s) about machine learning for security (K&U)
- Have acquired critical thinking around machine learning and security (MJ, LS)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

The course touches on various topics in machine learning and security. Examples of topics that can be included are: adversarial machine learning for image classifiers; Differential Privacy for distributed Stochastic Gradient Descent; secure multi-party computation for federated learning; the use of ML in security such as intrusion detection, analysis of silicon Physical Unclonable Function designs, GAN models generating synthetic data such that collected personal data can be discarded, and the moral character of cryptography, security, and artificial intelligence.

Additional Information Teaching Methods

Lectures (105 minutes 2x each week for 4 weeks); Group Q&A (135 minutes 1x each week for 3 weeks) during which (a) we cover lecture material for which we had no time during lectures, (b) upon student request re-explain topics and/or discuss upcoming assignments and (c) answer student questions about the lectured material and problems.

Method of Assessment

(1) At the start of the course you will write a short (but at least 1/3 page, single column) essay on your study strategy and your grade expectation:

You cannot pass the course if you don't submit by the deadline. And those who don't simply get NS "No Show" as the final grade.

(2) Questions and problems will be posted for each lectured topic. At the end of the course there will be a final in a controlled environment where a subset of the posted questions and problems for each topic will be asked. You will be able to prepare for this final by collecting and learning answers throughout the course. The final tests whether you have acquired a sufficient overall understanding. The final has a pass/fail grade.

If you do not pass the final, then you fail the course. If you pass the final, then your final grade is determined by your individually graded assignments as described below.

There is no re-sit opportunity for the final. A grade for the final from a previous year does not transfer to this year.

(3) There will be six (A = 6) individually graded assignments in the form of research surveys/essays together with your own self-evaluations of your submitted works. Each assignment is assessed based on the quality, correctness, and completeness of the solution(s) or written exposition. Each assignment is graded as a Fail (F), Pass (P), or Excellent (E).

(An F is also given for an assignment if you do not meet its deadline for submission.)

The following explains in text a rather complicated and intuitive formula for computing the final grade. A visual picture will be presented at the start of the course (the editor does not allow it to be inserted here).

We set

A = # Assignments

and we compute

N = 2*(#P) + 3*(#E),

where #P indicates the number of Passes you have received and #E the number of Excellent. (Notice that A = #P + #E + #F.)

If you have submitted your study strategy essay by the deadline and if you passed the final, then your final grade is equal to

14/6 + (17/6)*(N/A) rounded to the nearest integer in {1,2,3,...,10}.

The specifics of this grading policy are subject to change if the need arises.

There are no re-sit opportunities for any of the assignments/essays. Grades for assignments from previous years do not transfer to this year.

(4) At any moment during the course the instructor can request a student to explain one or more of his/her assignments in order for the instructor to verify whether the student understands the material presented in the

assignments; if the student is not able to explain his/her understanding in an assignment, then the grade for the assignment will be downgraded to a fail.

Entry Requirements

The course attempts to be self-contained and explains the necessary machine learning concepts as well as security and crypto techniques. Nevertheless, students are required to be independent and study extra where their mathematical background is limited. For example, you will want to know how to compute gradients, know about the structure of a proof, can read mathematical notation, etc.

Literature

Slide decks, papers, and reference material where needed.

Additional Information Target Audience

Master Computer Security; Master Computer Science; Master Artificial Intelligence

Security Experiments and Measurements

Course Code	XM_0098
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. ir. F. Massacci
Examiner	prof. dr. ir. F. Massacci
Teaching Staff	mr. S. Zhang MSc, prof. dr. ir. F. Massacci, W.B. Mbaka, dr. M. Zhang, S.A.M. van Gerwen, mr. E. Mezzi
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

“We don’t want high-level executives discussing whether a blue background or a yellow background will lead to more ad clicks. Why debate this point, since we can simply run an experiment to find out?” Hal Varian, Google Chief Economist [1]

The objective of this research design course is to introduce students to the critical issues behind the practical design, execution, and measurements of (security) experiments.

During the course students will

1. learn the ***scientific experimental methodology*** to determine the actual success of a security technology when applied by humans or in case control studies
2. learn some ***advanced security technologies*** (e.g. identification of vulnerability in codebases by using AI, decision making in threat intelligence; modelling threat in information systems, and one additional topic such as security in the cloud);
3. ***apply the technology*** some concrete studies as experimental subjects;
4. ***analyze the results of the experimental studies*** as security researchers to determine if such technology actually works.

The course is organised along the principles of ***scientific peer reviews*** (double blind) and expects motivated students to reach the ***highest educational objective*** in Bloom’s cognitive taxonomy (***Judgement***).

Course Content

Regular and active participation in the teaching activities offered by the course (lectures, experiments and in independent study activities) will enable students to:

- understand a specific security problem and try to address it in a laboratory case study ***[Applying knowledge and understanding]***
- understand the fundamentals of the design of (security) experiments involving both humans and code ***[Applying knowledge and understanding]***
- determine measures for the impact of a treatment and evaluating its success ***[Applying knowledge and***

understanding, Making judgements]
- identify and review pros and cons on the concrete problem instance by reviewing the work of their peers ***[Making judgements, Communication, Lifelong learning skills]***
- defend in public the design of an experimental security analysis***[Applying knowledge and understanding, Lifelong learning skills, Communication]***

Additional Information Teaching Methods

The course is organised into weekly lectures coupled with practical assignments where the students will apply gained theoretical knowledge on one or more case studies of a software system.

- ***Lectures and discussion sessions*** will introduce the students to four security technologies, show how they can be applied to some scenarios of moderate complexity, and provide additional material to study.
- ***Experiments in class in presence.*** Each student will be asked to apply the security technologies on which s/he has received training on some concrete scenarios of moderate complexity. Students should demonstrate that they are able to understand the scenarios.
- ***Scientific Reports.*** Each student will write and submit some scientific reports to analyse the results of the experiments.
- The ***first version of the reports*** will include an analysis of the performed experiments as in scientific articles
- The ***revised report(s)*** include a rebuttal of the received reviews on the previous version as in scientific articles
- Students who do not submit by the deadline will not be able to review the reports of their peers.
- ***Individual peer reviews:*** each student will review the assignments submitted by their peers according to a detailed grading scheme provided and exemplified by the lecturers
- Each student will have to review five assignments for each type of submission.
- The feedback of students of past editions of the course is that the average time for peer evaluation is estimated at ***one hour per (good) review.***
- ***Class presentation and public review.*** To provide intermediate feedback and to learn to identify and review a variety of applications of the technique students will be asked to discuss their assignments in front of the class.
- Attending the discussion classes is ***mandatory***, also for students that have already discussed in previous sessions.
- **Q&A sessions** (online) will be organized to answer questions.

The final list of experiments will be determined before the course starts. The following experiment happened in the previous years:

- ***Modelling Threats:*** students playing the role of analysts will have to decide whether it is easier to identify a risk by using a graphical model or by receiving textual information from a Large Language Model;
- ***Finding vulnerabilities in code:*** students playing the role of developers will try to identify vulnerabilities in a large file without support or when given a subset of the lines of code as selected by a ML/Static analysis algorithm;
- ***Threat Intelligence:*** students playing the role of intelligence analysts will have to decide whether the intelligence received by an AI algorithm is more credible than the one received by a human expert;
- Make your own: an experiment proposed by the students themselves.

Method of Assessment

Given the tight timeline, at the ***beginning of the course*** students will have to submit a short (1/3 page) essay on their study strategy and grade expectation and participate in a simple online experiment on threat assessment. Those that do not submit by the deadline will get a NS "No Show" as the final grade.

The ***overall grade*** is determined as follows:

70% for the ***Research Reports*** each of which is divided as

- 65% on the evaluation of the report as graded by the peers
- 35% for the participation in the peer review process and the quality of the reviews as re-evaluated by the lecturers and teaching assistants

25% for the ***Active Participation in Experiments***

5% for the design of a "Make Your Own"***Experiment Proposal***

Submission of Intermediate Results

- Each report is submitted weekly by a deadline. If a student does not submit a report this specific report will be counted a zero point
- If a student does not submit his/her own report, it cannot do the reviews of the other students reports

- Reviews are also submitted weekly by a deadline, every review that is not submitted by the deadline will be counted zero points.
- If a student submit some of the reports, or participates to some of the experiments and cannot reach 5.5 it will be marked NVD
- The only way to obtain a NS is not to participate to any activity.

Pass or fail

- ***Quality of the reviews.*** Additional points beyond the participation will be deducted for students who write poor (e.g. generic such as "it is ok, full points") or unfair reviews.
- ***Presentation of the reports in class***. If a student is asked to present his/her report in class and is absent , do not respond, or not able to do present it ("I do not remember what is in the report I submitted yesterday"), the report will be considered void.

Plagiarism and Fraud Check

If the number of students is so large that discussions in front of the class cannot be organised for all students to present at least once, a digital exam will be organised to assess the knowledge of the reviewed material. Students who have not presented must score ***at least 6/10*** in the confirmation exam for the assignments score to be considered valid.

Resit

It is not possible to resit the assignments and the peer reviews. Students can only resit failed reviews due to additionally deducted points by writing a report analyzing the reviews of all other students for each failed report.

Carry on of previous results from previous years

- This is generally not possible as the reports are delivered in installments and each report must address in the subsequent installment the peer review comments of the students of the course with the exception below:
- If a report and its peer review process and revision are entirely completed (Report submitted, peeer review done, report revised, peer review done) then it is possible to carry on the result provided the report has not being voided due to plagiarism/failure of discussion
- If the experiment is the same, students can recycle and adjust the same text and scripts of their own report from the previous year by simply updating the results with the new data.
- it is only possible to carry on the participation to the experiments if one has participated to all experiments the previous year.

Entry Requirements

See recommended knowledge.

Literature

Assumed pre-requisite

- Alan Agresti, Christine A. Franklin, Bernhard Klingenberg. Statistics: The Art and Science of Learning from Data, 4th Edition. Pearson Education.
  This is a classic textbook of statistics, typically used in BSc courses. Chapters 4 and 5 contain a useful summary that defines experiments, case-control studies, and prospective studies. Any other statistics text will work.

Necessary Material

- Lectures on security topics material, training for the experiments etc. will be available on CANVAS.

Useful

- Andrew Jaquith . Security Metrics: Replacing Fear, Uncertainty, and Doubt. Wiley.
  This is a classic book on measuring security. It is almost 13 years old and written a bit as a set of anecdotes. Still, it covers essentially all metrics you'll find in the industry and a rough description of all processes. We will use in particular the chapter on visualizing security metrics
- Rachel Glennerster and Kudzai Takavarasha. Running Randomized Evaluations: A Practical Guide. Princeton University Press.
  This is not a book about computer security. It is a book about social development and evaluating the impact of "interventions" (aka experiments) on poverty eradication. However, if one does a "search-and-replace" of the words "deworming program" with "vulnerabilities finding program" and the word "children" with "developers" it provides a good discussion of all problems that a security and computer science experimenter faces and how to solve them.
- Judea Pearl, Madelyn Glymour, Nicholas P. Jewell. Causal Inference in Statistics - A Primer. Wiley

A thin booklet on the notion of causality, and how we can formally capture it. We will use causal graphs to visualize the concepts behind our case studies. We will use the first two chapters.
- Cited [1] Michael Luca and Max H. Bazerman “Want to Make Better Decisions? Start Experimenting”MIT Sloan Management Review June 04, 2020. Interview with Hal Varian.

Additional Information Target Audience

Master Computer Science
Master Computer Security
Master Artificial Intelligence

Additional Information

"***Participating as a subject***" means that each individual student will read or watch a training specific to the task (e.g. how to identify a Java vulnerability on source code), then will be assigned a task (e.g. find vulnerabilities in the following two random files, one filtered by a Machine Learning tool a second just the raw code from a file), and execute the task to the best of one's ability within a time limit (e.g. report the vulnerable lines within 2 hours).

"***Analyzing the results***" means that each student will receive all data collected in the previous steps (plus some other additional data) and will have to assemble a report describing the experiment so that other people could replicate it, the results of the experiments in their opinion (e.g. plots showing that there was no difference between the file processed by the ML and the unaided human, statistical analysis, etc.) and what the criticality and threats to validity of the experiments were (now directly from their own experiments).

All reviewers are anonymous (to the peers) but not to the teachers who act as program chairs in a scientific conference.

Custom Course Registration

Given the tight timeline, at the beginning of the course students will have to submit a short (1/3 page) essay on their study strategy and grade expectation and participate in a simple online experiment on threat assessment. Those that do not submit by the deadline will get a NS "No Show" as the final grade.

Explanation Canvas

The submission of the report and the peer review of the reports will be done in Canvas/FeedbackFruit.

Because of the way the system works by automatically assigning reviewers, it is not possible to grant any deadline extension for the submission of the reports and the submission of the reviews.

Students should therefore carefully plan their activities and submit also a draft version of their reports well in advance of the deadline (multiple submissions are possible) rather than waiting for the last moment.

Recommended background knowledge

Basic general knowledge of statistics at BSc level are assumed (mean, standard deviation, concept of distribution and confidence interval, and simple statistical tests). A basic data analysis code will be supplied by the teacher.

Good working knowledge to write scripts or programs to manage data and software repositories is useful to avoid manual analysis. Scripts for analyzing data can use Matlab, R or Python. Use of such languages is up to the students.

Good analytical knowledge of programming languages (e.g. Java, C, Python,..) and software testing is key to being able to perform the analysis of code vulnerabilities. Introductory knowledge of software security is also assumed.

Service Oriented Design

Course Code	X_405061
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J.R.E. Bogner MSc
Examiner	dr. J.R.E. Bogner MSc

Teaching Staff	prof. dr. P. Lago, dr. J.R.E. Bogner MSc
Teaching method(s)	Seminar, Lecture

Course Objective

Upon completion of this course, students will be able to:

- Analyze a design problem and model software for it in terms of independent services by using notations from industrial practice (Knowledge and Understanding; Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).
- Apply structured techniques to make and justify design decisions for service-oriented software (Knowledge and Understanding; Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).
- Navigate quality and sustainability requirements and understand how to achieve them in service-oriented design (Knowledge and Understanding; Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).
- Collaborate in a team and synchronize their project contributions into one holistic presentation (Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).

Course Content

Throughout the course, students will work in teams to provide a solution for a case presented by invited guests from an external company or organization (case providers). Students will analyze the problem domain and design a service-oriented solution for it.

For this project, the lectures explain the required theoretical concepts about service orientation as a software paradigm, as well as the difference between service-oriented architecture and microservices. More importantly, the lectures introduce techniques for identifying the requirements for service-oriented software, how to map them to business services, and how to transform them into complex networks of software services. Several UML-based modeling techniques and diagrams are used in the course. Special emphasis is given to design reasoning techniques for decision-making, service identification, service-oriented software design, and sustainability concerns. Lastly, the course also touches on RESTful API design and service-oriented design patterns.

Additional Information Teaching Methods

The course consists of three live sessions per week (each 01:45 h long):

- Lectures: acquire necessary theory and understand used modeling techniques
- Project sessions: discuss your team progress with your TA and other teams (mandatory attendance)
- Presentation sessions: present your project to all other teams in a plenary session

Method of Assessment

The grade is mostly determined by two team-based and project-based assignments (A1 and A2). A1 focuses on the selection and analysis of the design space for the chosen project. A2 revises and extends A1 and proposes a service-oriented design solution.

Individual contribution to the project:
Each assignment has parts that count for the whole group and parts that count only for an individual student. In general, the project workload should be distributed fairly among the group members. Insufficient contribution to the group parts of the assignment can lead to failing the course. The final grade is the average of both assignment scores, plus potential bonuses (see below).

Project presentation:
In addition to the individual sections per assignment, each student is expected to both attend and contribute actively during the weekly **project sessions** with the TA. Attendance is documented, and missing too frequently will lead to failing the course. Moreover, each student needs to fulfill two roles in the controlled environments of these sessions at least once over the course of the project.

- **Presenter:** present the progress of your team and answer questions
- **Discussant:** moderate the discussion following another team's presentation and ask some questions yourself

Each of these roles will be assessed with either *fail*, *pass*, or *pass and +0.15*.
Lastly, each student also needs to have an active part in the **plenary presentation session** when their team is presenting to everyone. This part is also assessed with *fail, pass*, or *pass and +0.15*.
The three potential bonuses (up to a maximum of +0.45) will be applied to the average of the two assignment scores. Receiving a *fail* in any of these three roles will lead to failing the course.

Canvas quizes in lectures:
In each lecture, a Canvas quiz takes place that can provide a bonus to the final grade (+0.04 per quiz, for a total of +0.28). These Canvas quizzes do not impact the grade negatively.

Final grade:
To pass the course, a student must meet the following criteria:

- Contributed adequately to the group parts of A1 and A2
- Did not miss more than 2 project sessions with the TA
- Did not receive a *fail* for the project presenter role
- Did not receive a *fail* for the project discussant role
- Did not receive a *fail* for presenting in the plenary presentation session
- Both A1 and A2 have a score of at least 6.0

If these criteria are fulfilled, the final grade is calculated as follows:
(A1 score + A2 score) / 2 + project presenter bonus + project discussant bonus + plenary presenter bonus + Canvas quiz bonus
This number is then rounded to the nearest 0.5, e.g., an 8.67 is rounded to 8.5, while an 8.81 is rounded to 9.0.

Mode of re-examination:
Each failed project assignment (A1 and A2) can be revised and resubmitted once (*resit*). Failing the other mandatory contributions can be recovered in the following ways:

- Missed more than 2 project sessions with the TA without reasonable explanation, e.g., medical issues: take over additional shared project-tasks from your team members to make up for your missed attendance
- Received a *fail* for the presenter role: privately presenting the project to the lecturers
- Received a *fail* for the discussant role: acting as discussant in one of the plenary presentation sessions or privately discussing your project with the lecturers
- Received a *fail* for presenting in the plenary presentation session: taking over the presenter role for your team more often in the project sessions or privately presenting the project to the lecturers

Literature

All course material will be distributed online on Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Custom Course Registration

Further information for this course will be made available on Canvas, in which all students must be enrolled. Group enrollment also takes place via Canvas and a shared spreadsheet.

Explanation Canvas

Instructions about group enrollment and the course process will be provided at least one week before the course starts. All students must be enrolled before then.

Recommended background knowledge

- At least intermediary software design and modeling knowledge
- At least intermediary knowledge of the Unified Modeling Language (UML)
- At least intermediary programming skills, even though no code will be written in the projects

Software Security

Course Code	X_400127
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. E. van der Kouwe
Examiner	dr. E. van der Kouwe

Teaching Staff	prof. dr. ir. H.J. Bos, dr. E. van der Kouwe
Teaching method(s)	Lecture

Course Objective

At the end of the course students will:

- have knowledge of the concept of vulnerability in design and system implementation of applications. (K&U)
- (by doing) deeply understand the concepts of memory corruption attacks (such exemplified by buffer overflows, use-after-free, etc), (K&U, AK&U, MJ)
- (by doing) deeply understand mitigations (such as control flow integrity, address space layout randomization, and shadow stacks) as well as some of their weaknesses (K&U, AK&U, MJ)
- have hand-on experience with basic and advanced exploitation and mitigation methods, including state-of-the-art research in systems security (K&U, AK&U, MJ)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This is a course on security with a focus on systems work. The course covers a wide spectrum of security issues. We explicitly focus on systems security rather than cryptography, as we want to show students how attackers penetrate systems and most security issues are not due to bad crypto.

Specifically, the course focuses on memory corruption and application security (buffer overflows, format string bugs, dangling pointers, shellcode, return-oriented programming, ASLR/DEP/canaries, control flow integrity), and state-of-the-art attacks and defenses (latest research insights).

Much of the course will be hands-on and challenge-based. In assignments, students will carry out and investigate attacks in a controlled environment. This involves programming at the lowest level (in assembly).

Additional Information Teaching Methods

Lectures and (very challenging) practical assignments.

Method of Assessment

Written exam (30%) and practical assignments (70%). The exam must be passed with a grade >= 5.5. Each of the practical assignments must be passed with a grade >= 4.0. The final assignment grade is the weighted average of the assignments' grades. Each assignment must be completed individually.

Please note that your grade will be registered as follows:

- For the first exam date: if you did not submit any assignments and did not participate in the exam, the result will be registered as no show (NS).
- For the second exam date: if you did not participate in the resit exam, the result will be registered as no show (NS).
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments and/or the exam, but your weighted average is >= 5.5, your grade is NVD.
- Otherwise, your final grade is your weighted average, rounded to half points except for 5.5.
- If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

There is no resit opportunity for the practical assignments. Results for practical assignments and for the exam may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner). If you submit an assignment or participate in the exam, the new result replaces any transferred results.

We ensure that each student individually demonstrates their abilities through the requirement that the supervised exam be passed with at least 5.5 and through active plagiarism detection for the assignments.

Literature

No set book. All material will be made available during the course.

Additional Information Target Audience

Master Computer Security
Master Computer Science

Recommended background knowledge

No formal requirements, except a keen interest and sufficient time.
Programming experience in C is essential. Knowledge of assembly and computer architecture helps too.

Software Testing

Course Code	X_400439
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. N. Silvis-Cividjian
Examiner	dr. N. Silvis-Cividjian
Teaching Staff	dr. N. Silvis-Cividjian
Teaching method(s)	Practical, Written partial exam, Lecture

Course Objective

After completing this course, students will be able to:

1. Appropriately use terms specific to the testing process. (Knowledge and understanding) (Applying knowledge and understanding)
2. Analyze a software-related failure accident and report on it. (Applying knowledge and understanding) (Making judgements) (Communication)
3. Given a software requirements specification (SRS) and an existing software product that can be standalone or embedded in a device, design and implement a comprehensive test strategy and produce testing documentation according to modern software engineering standards. (Applying knowledge and understanding) (Making judgements) (Communication) (Lifelong learning skills)
4. Practice software testing in a professional, responsible, and ethical manner. (Lifelong learning skills) (Making judgements)
5. Identify possible discrepancies between software testing as taught in an academic context and software testing as it happens in real IT companies. (Knowledge and understanding) (Lifelong learning skills)

Course Content

Testing is a method to improve software quality. Realistically, software testing is a trade-off between budget, time, and quality. It is impossible to test everything, so choices have to be made. Students learn how to make and justify these choices and systematically test a software product (standalone or embedded in a smart system).

This course teaches students how to test software in the broad context of systems engineering. It provides an introduction to software testing with an emphasis on technical activities like test generation, selection, execution, and assessment, and on non-functional requirements such as safety and security.

The course tries to answer a few questions, such as:

- How to learn from accidents?
- How to design adequate test cases in a limited time?
- When to stop testing?
- How to test a safety-critical software-intensive system?
- What are the challenges of embedded software testing?

A few guest lectures showing examples of testing in the industry area also planned.

Topics: boundary value analysis, equivalence partitioning, combinatorial testing, model-based testing, control-flow testing, data-flow testing, mutation testing, regression testing, inspections, automated testing, integration testing, system testing, hazard analysis, fuzz testing, diagnostic tools for embedded systems testing.

Additional Information Teaching Methods

Lectures and mandatory practical.

Method of Assessment

The GROUP work is graded with a pass/fail and is evaluated through a few face-to-face interviews with the teaching team. The Individual component has two parts, both proctored: a bug hunting exam in a tool called DBugIT and an exam in Testvision. A pass for the GROUP component and a grade of >= 5.5 for both exam components are needed to pass the course.

Literature

- A. Mathur, Foundations of software testing, Addison-Wesley Professional; 2nd edition (February 13, 2014), 2014, *ISBN: * 978-8131794760.
- Lecture notes, and author's textbook (in press).

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Computational Science

Explanation Canvas

All course materials are available on Canvas.

Recommended background knowledge

Basic knowledge of programming (Python, C++) and software engineering (requirements engineering) is required.

Systems Seminar

Course Code	XM_0122
Credits	6
Period	P4
Course Level	600
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	B. Chandrasekaran
Examiner	
Teaching Staff	dr. D. Bonetta, dr. T. De Matteis, B. Chandrasekaran
Teaching method(s)	Seminar, Practical, Lecture

Course Objective

The key objective of this seminar is to introduce students to designs, details, and challenges associated with building modern computer systems (CompSys) from a single machine to distributed services. As part of this exercise, students will read, review, present, and discuss recently published cutting-edge research on the design and implementation of computer and networked systems.

The course takes the form of an interactive seminar (presentations and discussions) mostly driven by the students. Upon completion of this course, students will be able to:

- Reason about various design choices, systems building trade-offs, and ongoing and emerging research challenges for a design based on the scale, deployment goals, and capabilities provided. [Applying knowledge and understanding] [Making judgements]
- Understand recent advancements in systems (leveraging modern hardware - SmartNICs, fast storage, FGPAs, programmable switches) and cutting-edge techniques to run these systems safely and efficiently. [Knowledge and understanding]
- Read papers, write critical (technical) reviews (style, tone, details), compare details across other papers, understand how to evaluate a paper, and develop and learn about research methodology skills. [Knowledge and understanding] [Applying knowledge and understanding] [Communication]
- Learn how to present and lead discussions on state-of-the-art literature in an interactive setting with your peers [Communication] [Lifelong learning skills]

Course Content

The course is largely student-driven. Each week we will have paper presentations and discussion sessions where we will discuss recently published, high-impact papers in the literature. We will provide a curated list of recently published papers in the first week of the class to cover these topics. The acquisition of this knowledge is important to understand how the fields of computer systems are emerging, and how future cloud/distributed systems in practice will be shaped.

In each class session there will be a presentation(s) from students, and students will lead the discussion on the topic.

Outside the class, the students need to read papers and write detailed technical reviews.

Additional Information Teaching Methods

In each lecture session we will discuss a few closely related papers in detail. Every student is expected to read all papers and send weekly reviews before the week starts. A sample format for the review and previous year's review samples will be provided.

There will be no deadline extensions and we will not allow late submissions.

In each lecture there will be presentations (15-20mins) from students, and then an open, conference program committee (PC) style discussion led by students and teachers regarding the (i) content and contributions; (ii) strengths; and (iii) weaknesses (and what could be improved); (iv) how papers which are discussed are related to each other; (v) various other issues raised in your submitted reviews; and (vi) overall broader research direction to which the papers belong. We expect the students to actively participate in these presentations.

Method of Assessment

Students are awarded points every week on the quality of the review (technical correctness, understanding of the paper, details covered, tone and style), and the presentations (style, delivery, visuals, and audience engagement). The final grade is a weighted average of points: Reviews (60%) and presentations (40%).

Reviews: The papers reviewed will be from one or more distinct research areas covered in the course. You must score at least 5.5 in the reviews (in total) and 5.5 on average in the reviews belonging to each of the different research areas.

Presentation: Each student must present a paper, failing which you will not be able to pass the course; there is no minimum score or cutoff for the presentation though.

- The final grade is calculated as a weighted sum of points obtained for the reviews and presentation, as shown above.
- There are no resits available for this course.
- This course does not allow students to carry over (partial) grades from last year.

[Fraud] The course is made fraud resistant by having selective and specialized reviews to assess the knowledge of the students and deep technical presentations, both of which are graded individually.

Please note that your grade will be registered as follows:

If you do not submit any review and do not present a paper, you will receive a NS.
In all other cases, you will receive either a numeric grade or incomplete (NVD).
If you did not meet the minimum grade requirements for the reviews, your grade will be NVD.
Otherwise, your final grade is your weighted average.
If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

Literature

A list of recently published and classic papers (mandatory and background readings) reflecting recent developments.

A sample format for the review and previous year's review samples will be provided. We will also introduce advanced topics, reproducibility research methodology, and tips for reviewing and presenting works.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

This course builds on a wide variety of computer systems and networking knowledge and concepts such as design

of operating systems, networking and storage stacks, integration of accelerators, memory virtualization and management, network architecture, Internet-scale computing, process management, failure and fault tolerance in distributed systems, computer architecture (interrupts, CPU caches, multi-core systems, I/O buses), etc. Due to the nature of the course, there will be limited opportunity to learn these concepts in the class. In case you lack necessary background knowledge, ask us early to provide background reading material that you should cover by yourself.

The Social Web

Course Code	X_405086
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. D. Ceolin
Examiner	dr. D. Ceolin
Teaching Staff	dr. F. Ilievski, dr. D. Ceolin, Z. Inayat
Teaching method(s)	Computer lab, Lecture

Course Objective

In this course, students will:

- Learn theory and methods concerning communication and interaction in a Web context (Knowledge and Understanding).
- Understand how user data is represented and distributed across devices in the context of the Social Web (Knowledge and Understanding).
- Collect and prepare Web data, apply the above skills in practical assignments (Apply Knowledge and Understanding)
- Reflect on social and ethical responsibilities linked to the application of these methods (Making Judgements).
- Communicate knowledge and judgements in a final paper (Communication).

Course Content

This course will cover theory, methods and techniques for:

- personalization for Web applications;
- Web user & context modeling;
- user-generated content and metadata;
- multi-device interaction;
- usage of social-web data.

The course takes a data science approach, using Jupyter notebooks to collect and analyse data, with a research paper describing the data science study as the main outcome of the course.

Additional Information Teaching Methods

Lectures (1x week)
Practical sessions (1x week)
Presentation seminars (1x week)

Method of Assessment

The weighted average of:

- 70% research paper (group assignment A),
- 10% group presentation (P),
- and 20% individual exam (E).

A minimum grade of 5.5 for each component is required.

Compensation: it is not possible to compensate one component with another.

Mode of re-examination: if insufficient, each component may be revised and resubmitted once.

Literature

- Course lecture slides;
- Selected articles, videos and Web links

All materials for each lecture will be provided through Canvas.

Additional Information Target Audience

Master Artificial Intelligence
Master Computer Science
Master Information Sciences
Master Computer Security

Additional Information

The course has a maximum enrolment limit of 120. The students of the Master Information Sciences, for whom it is compulsory, have priority over the students that take this course as an elective or a constrained choice course.

The students that take this course as an elective or a constrained choice course run the risk of not being able to participate, even if they have registered on time, if the maximum enrolment limit has been reached.

The students of the Master Information Sciences, for whom it is compulsory, run the risk of being initially refused if the enrolment limit has been reached at the time of registration. However this will be adjusted manually after registration has been closed so that all the students of the Master Information Sciences will be able to participate.

Recommended background knowledge

Basic Python programming skills.

Verification for Security

Course Code	XM_0099
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. K. Freiherr von Gleissenthal
Examiner	dr. K. Freiherr von Gleissenthal
Teaching Staff	dr. K. Freiherr von Gleissenthal
Teaching method(s)	Seminar, Lecture

Course Objective

In this course, students get acquainted with techniques for program verification with a focus on low user effort and automation and learn how to apply these techniques to prove that programs are secure.
After completing the course, students will:

- Have learned the basics of checking correctness proofs using SMT-solvers. (Knowledge and Understanding)
- Have learned how to encode program correctness proofs in SMT via Hoare Logic. (Knowledge and Understanding, Applying Knowledge and Understanding)
- Understand and know how to apply algorithms for computing correctness proofs automatically. (Knowledge and Understanding, Applying Knowledge and Understanding)
- Have learned how to use verification techniques to prove that programs are secure, that is, they don’t leak sensitive data directly, or indirectly through the cache, and timing. (Knowledge and Understanding, Applying Knowledge and Understanding)
- Have gained practical experience in applying these techniques. (Applying Knowledge and Understanding)
- Understand the limitations of verification techniques and gain an understanding of when to apply them. (Knowledge and Understanding, Applying Knowledge and Understanding, Making Judgements)

Course Content

When is a program correct? When is it secure? It is often hard to convince yourself that a program works for a given set of inputs, let alone for all possible inputs and without inadvertently leaking confidential information. In this

course, you will learn how to formally reason about---and prove---that a program is correct and secure. Since life is short and writing proofs is difficult, we will focus on low-effort techniques and try to let the computer automatically discover proofs for us. We will start with proving general program properties, say that a program never reaches a bad state, and then move on to proving security and confidentiality, more precisely, that a program does not leak secrets, directly, or indirectly via cache, timing, or micro-architectural side-channels like Spectre. We will study verification not purely in theory, but instead, we will implement verification methods and get hands-on experience applying them.

Additional Information Teaching Methods

Lectures and practical assignments.

Method of Assessment

Written exam (50%) and practical assignments (50%). The exam must be passed with a grade >= 5.5. Each of the practical assignments must be passed with a grade >= 4.0.

Please note that your grade will be registered as follows:

- If you do not submit any assignment and do not participate in the exam, you will receive a NS.
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments, your grade is NVD.

If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade. There is no resit opportunity for the practical assignments.

Results for practical assignments and for the exam may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner).

We ensure that each student individually demonstrates their abilities through the requirement that the supervised exam be passed with at least 5.5, through active plagiarism detection for the assignments, and by investigating whether each group member contributed sufficiently to the group assignments.

Literature

No set book. All material will be made available during the course.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

Students are expected to know functional programming. The assignments use advanced features such as typeclasses and monads.

Web Services and Cloud-Based Systems

Course Code	XMU_418110
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130392

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.